

# Observation-Stable Model Unweaving: Local Certificates and a Two-Write Boundary

HAOYI ZHANG, Xi'an Jiaotong-Liverpool University, China

HUAIJIN RAN, Nanyang Technological University, Singapore

Selective unweaving asks which aspect applications can remain after others are removed. An inverse log answers a different question: how to recover a particular predecessor state. We study their relationship for a finite, typed, fixed-match trace language with grouped aspect occurrences, unary observations, and constant overwrites. A last-occurrence normalization makes each individual observation a unate Boolean function of retention decisions. Its non-subsumed clauses are exactly its prime implicates, yielding cardinality-minimum local failure explanations and a precise flat-CNF size bound. Across observations, these local properties do not imply global minimality. We characterize the language's retention functions as conjunctions of mixed clauses with at most two negative literals, without auxiliary aspect variables. If each cell is written at most once, policy completion reduces to implication closure and every infeasible consistent policy has a two-fact minimum conflict. With at most two writes per cell, completion is NP-complete, even for Boolean cells, one event per aspect, and one forced retention and removal. Global conflicts can grow arbitrarily while all local clauses remain ternary. Complement receipts support round trips but must be regenerated after selective replay. A separate executable checker validates outcomes, local explanations, refutations, and minimum-policy conflicts. An exact small oracle and 50,000 bounded cases agree on 231,071 retention queries; additional structural checks and six source-guided projections test the stated boundaries. The proofs are mathematical arguments, not proof-assistant mechanization, and the evidence concerns the specified trace semantics rather than rematching or deployed model repositories.

Additional Key Words and Phrases: model transformation, selective undo, reversible computation, trace semantics, prime implicates, proof certificates, dependency analysis

## 1 Introduction

A woven model records a sequence of decisions: which matches were found, which observations justified them, and which changes their advice performed. Removing an aspect need not mean returning to an earlier point in that sequence. Other aspects may have to remain, including aspects that ran later or whose occurrences are interleaved with the removed aspect. The final state of the original execution is then not enough to describe the requested result. Nor is a list of inverses enough to say whether the retained applications still mean what they originally meant.

Consider a Boolean cell initially zero. Aspect  $a$  writes one, aspect  $b$  also writes one, and aspect  $r$  observes one. Removing either writer alone preserves the observation; removing both does not. A dependency that makes  $r$  require the original last writer  $b$  is unnecessarily restrictive. Now change  $b$  to write zero and  $r$  to observe zero. The full trace remains executable, but retaining only  $a$  and  $r$  fails. Removing an additional aspect can thus destroy an observation rather than merely eliminate a dependency. These are small examples of two separate phenomena: alternative providers and later writers that restore a condition after an earlier writer destroys it.

Both phenomena are old enough that novelty cannot lie in naming them. Aspect model unweaving already uses traceability, inverse operations, impact analysis, and separate independent, additive, and general cases. In particular, the general case can require rematching and reweaving, and the additive case can distinguish individual match occurrences of one aspect [Klein et al. 2009]. Reversible event structures also account for disjunctive causes and more expressive enabling and reversal conditions [Ulidowski et al. 2018]. A theory that merely attaches a partial order to an inverse log would miss behavior already recognized by these accounts. The question here is instead about

---

Authors' Contact Information: Haoyi Zhang, Xi'an Jiaotong-Liverpool University, Suzhou, China, [hyeliozhang@gmail.com](mailto:hyeliozhang@gmail.com); Huaijin Ran, Nanyang Technological University, Singapore, Singapore, [huaijin003@e.ntu.edu.sg](mailto:huaijin003@e.ntu.edu.sg).

an exact, restricted interface between a captured model-weaving execution and an independently checked selective replay.

### 1.1 A stability contract, rather than an unspecified inverse

We take the recorded match instances as fixed. Each event has an aspect tag, a finite conjunction of unary observations, and simultaneous constant overwrites. All occurrences carrying the same tag are retained or removed together. The remaining events execute in their original order. A retention is valid precisely when every retained event still satisfies its recorded observations. We call this *observation-stable unweaving*. A consumed value that determined a captured constant must be observed by equality; a pure test may record its full accepting set. This distinction permits exact reasoning about a threshold test without silently claiming that an arbitrary captured computation may change its inputs.

This contract is deliberately narrower than repeating source-level weaving. It neither discovers a new match that an erasure makes possible nor recomputes advice on a different input. Its attraction is that the success condition is explicit and decidable from the trace. A refusal has the equally explicit meaning that no retention satisfying a stated policy preserves all these recorded observations. It does not assert that no alternative model or source-level reweaving exists. The difference is essential in both the semantics and the source-guided examples.

The state includes a finite catalog of typed cells. Boolean presence cells encode nodes and edges with fixed endpoints, and a static event rule preserves the invariant that a present edge has present endpoints. Scalar cells contain finite attributes. The catalog is fixed, so allocation is represented by changing presence rather than by generating a fresh identity. This is enough to express finite creation, deletion, property changes, and incidence-sensitive examples, while keeping the semantic argument separate from graph matching and name generation.

### 1.2 What information does the trace need?

There are two different information problems. To reverse an already executed constant overwrite, the erased old value must either be inferred from the event's observations or stored as a complement. Such complements are a familiar bidirectional-transformation idea, not a new account of lossless information recovery [Hofmann et al. 2011]. To decide a selective replay, one also needs to know what retained events observed and the order of relevant writes. Two traces can have the same final state, named writes, and old-value receipts, yet answer a retention query differently. We give explicit indistinguishability pairs rather than claiming that every field of an uncompressed trace is always necessary.

Even a valid selective replay can invalidate the original receipts. With a three-valued cell initially zero, let  $a$  write one and  $b$  write two. Retaining only  $b$  is executable. Its original receipt, however, stores one, while the correct predecessor in the selected replay is zero. An end-state comparison sees no difference: both end at two. We therefore distinguish a valid retained execution from a reversible retained execution whose complements have been regenerated and checked against its actual pre-states.

The main information result is local. For a fixed observation, group all writes by aspect, discard writes before a forced reader-tag anchor, and keep only the last remaining write of each other tag. Each tag then has one role: it provides an acceptable value or an unacceptable value. This makes the observation's Boolean retention function *unate*: every variable has one polarity after compilation into clauses. Established prime-implicate reasoning then gives exact local minimum explanations [Darwiche and Hirth 2023; Darwiche and Marquis 2002]. The trace-specific step is the normalization that exposes this structure despite repeated, interleaved occurrences. It is not a claim to invent Boolean explanations.

### 1.3 Local structure and global difficulty

The local normal form does more than support diagnostics. Every local clause has one negative reader literal, at most one additional negative writer literal, and at least one positive repairing-writer literal. Conversely, any conjunction of clauses of that form can be realized by a Boolean trace, using fresh cells and the original visible aspect variables. Thus the trace language has an exact Boolean expressivity boundary without hidden aspect decisions. Being true at the empty and full retention is necessary, but not sufficient, for expressibility.

A second boundary concerns the number of writes to each cell. With at most one write per cell, every nontrivial condition is a binary requirement: retaining a reader requires retaining the unique writer. Policy completion then has a unique maximum solution, found by backward propagation from forced removals. If a consistent policy is impossible, one forced retention and one forced removal already constitute a minimum conflict. With at most two writes, this picture changes. Policy completion is NP-complete even with Boolean data, one event per aspect, and just one forced retention and one forced removal. In that same fragment every local prime has at most three literals. Small local explanations therefore do not imply easy global policy search.

The distinction also affects the word “minimum.” A smallest set of decisions that forces *one observation* to fail can be larger than a smallest set that makes *the whole replay* impossible. Different completions may fail at different observations. A binary choice-tree construction amplifies this separation: all local clauses are ternary, but a minimum global policy conflict has arbitrarily many facts. The certificates and checker expose these quantifier differences rather than reporting one undifferentiated kind of residual.

### 1.4 Claims and evidence

The technical claims concern the following connected argument. Typed events preserve graph presence and admit complement-backed round trips. Their selective replays factor exactly into ghost observations. Last-occurrence normalization produces each observation’s complete local prime representation, with an exact minimum-fact explanation and a representation-qualified size bound. The resulting clause language determines both an expressivity boundary and a sharp one-write/two-write tractability threshold. A separate certificate semantics checks successful outcomes, global refusals, minimum collateral removals, and cardinality-minimum conflicting subpolicies.

The implementation separates the producer, the direct finite oracle, and a checker that imports neither the producer nor its normalization. The main campaign contains 50,000 bounded inputs and 231,071 retention queries; every query agrees with direct replay, and each input has a checked outcome or local certificate. Separate tests cover exact local primes, small typed updates, commuting events, inconsistent certificates, SAT and vertex-cover constructions, and the stronger expressivity and writer-count results. Six source-guided micro-models are reported as projections, not as thirty public benchmarks or as executions of existing weaving tools.

These are complementary forms of evidence. Mathematical induction and reductions support unbounded finite statements. Executable enumeration tests bounded implementations and counterexamples. A separate code path reduces shared implementation assumptions but is not a proof-assistant development or independent human review. The article’s applicability rests on the declared trace interface, not on a case count standing in for model-management deployment.

### 1.5 Organization

Section 2 defines the typed trace and complement semantics. Section 3 derives exact observation conditions and their normalization. Section 4 proves the local minimum claims and separates them from global explanations. Section 5 establishes expressivity, dependency, commutation,

and complexity boundaries. Section 6 defines the certificate judgments and the separate checker. Section 7 presents the finite evidence with its population and resource limits. Section 8 relates the guarantees to the closest semantic and explanation interfaces and states the remaining scope limits.

## 2 Typed Captured Events and Complement-Backed Round Trips

The calculus describes an execution that has already resolved its source-level matches. This section makes that restriction operational: the event syntax states all observations, the typing rule states the preserved graph invariant, and the receipt rule states what an inverse may use. None of these judgments assumes that a producer's informal account of an aspect is trustworthy.

### 2.1 Signatures, states, and aspect grouping

A signature  $\Sigma$  consists of a finite cell set  $X$ , a nonempty finite domain  $D_x$  for each  $x \in X$ , a set  $N \subseteq X$  of node-presence cells, and a disjoint set  $E \subseteq X$  of edge-presence cells. Presence domains are Boolean. Each  $e \in E$  has fixed endpoints  $s(e), t(e) \in N$ . Other cells represent scalar attributes. A state is a total valuation

$$\sigma \in \prod_{x \in X} D_x.$$

It is well typed, written  $\Sigma \vdash \sigma$ , when

$$\forall e \in E. \quad \sigma(e) = 1 \implies \sigma(s(e)) = \sigma(t(e)) = 1. \quad (1)$$

An edge has its own presence cell even when another edge has the same endpoints. Self-loops are allowed. Cells are identities, not values to be equated modulo a graph isomorphism. Scalar storage remains allocated when a node is absent; an application needing a different storage discipline must specify an additional semantics rather than assume one here.

Let  $\mathcal{A}$  be a finite set of aspect tags. A captured event is a triple  $e_i = (a_i, g_i, w_i)$ , where  $a_i \in \mathcal{A}$ ,  $g_i$  is a finite map from cells to nonempty subsets of their domains, and  $w_i$  is a finite map from cells to domain members. Write  $R_i = \text{dom}(g_i)$  and  $W_i = \text{dom}(w_i)$ . A cell may occur in both maps: it is observed in the pre-state and then overwritten. The observation condition is

$$\sigma \models g_i \iff \forall x \in R_i. \sigma(x) \in g_i(x).$$

The event's update is simultaneous:

$$\sigma[w_i](x) = \begin{cases} w_i(x), & x \in W_i, \\ \sigma(x), & x \notin W_i. \end{cases} \quad (2)$$

The execution rule is  $\sigma \xrightarrow{e_i} \sigma[w_i]$  when  $\sigma \models g_i$ . There is no expression evaluation inside  $w_i$ . In particular, an update informally described as “increment  $x$ ” must have been captured with a singleton observation of  $x$  and its resulting constant, or must be excluded. Omitting that read would change the contract.

A trace  $T = e_0 \cdots e_{n-1}$  is admitted relative to base  $B$  only if  $B$  and all events are well typed and the full trace executes from  $B$ . Its original intermediate states are  $\sigma_0 = B, \dots, \sigma_n$ . A retention is a set  $K \subseteq \mathcal{A}$ . Its selected trace  $T \upharpoonright K$  preserves event order and includes exactly occurrences with  $a_i \in K$ . Tags, rather than occurrence positions, are the policy units. Two occurrences with one tag cannot be retained independently merely because they have different match sites.

**Definition 2.1** (Observation-stable retention).  $\text{Valid}_{B,T}(K)$  holds when  $T \upharpoonright K$  executes from  $B$ . Write  $\mathcal{V}_{B,T} = \{K \subseteq \mathcal{A} \mid \text{Valid}_{B,T}(K)\}$ . The empty retention and the full retention belong to  $\mathcal{V}_{B,T}$ : the former executes no event and the latter is part of trace admission.

The base is fixed across queries. The original final model may be delivered with a trace, but it is not an input needed by this definition. Replaying from the base gives a semantic reference point independent of any optimized unweaving strategy. It is a baseline for correctness, not a claim that rebuilding every large model would always be efficient.

## 2.2 What does an observation preserve?

An observation of  $x$  in  $S$  preserves membership in  $S$ , not necessarily the identity of the value. For example, over  $D_x = \{50, 150, 200\}$ , the test  $x \geq 100$  is represented by  $S = \{150, 200\}$ . Removing a write of 200 can leave 150 without violating the test. That does not justify retaining an advice constant whose computation depended on the exact number 200. Such an advice must instead record  $S = \{200\}$  for that consumed input. The event interface therefore distinguishes a branch condition from an exact input dependency.

The distinction is semantic, not a compression heuristic chosen after a query. For a fixed captured event, its sets are part of the input contract and do not expand to make a requested erasure succeed. A trace with singleton observations and a trace with wider sets are different scientific inputs. Comparing their answers illustrates the influence of the observation specification, but is not an optimization that preserves one fixed specification.

The syntax admits conjunctions of unary tests, not arbitrary predicates over several changing cells. The equality  $x = y$  generally cannot be expressed by such a conjunction. Recording the two original values as singletons yields a stronger stability contract, not an equivalent encoding of equality. This restriction is central to the local analysis: once the last retained writer of one cell is known, one observation can be evaluated without considering another cell.

## 2.3 A sufficient static rule for graph presence

For an incidence  $(e, p)$  with  $p \in \{s(e), t(e)\}$ , define the predicates

$$\begin{aligned} \text{zero}_e(g, w) &\iff (e \in \text{dom}(w) \wedge w(e) = 0) \\ &\quad \vee (e \notin \text{dom}(w) \wedge g(e) = \{0\}), \\ \text{one}_p(g, w) &\iff (p \in \text{dom}(w) \wedge w(p) = 1) \\ &\quad \vee (p \notin \text{dom}(w) \wedge g(p) = \{1\}). \end{aligned}$$

An equality involving  $g(x)$  is false when  $x \notin \text{dom}(g)$ . The typing rule requires all written constants and observed members to have the declared domain. For each incidence on which at least one of  $e, p$  is written, it additionally requires

$$\text{zero}_e(g, w) \vee \text{one}_p(g, w). \quad (3)$$

If neither cell is written, the old incidence implication is inherited. This rule is syntactic and local to the event: it does not inspect a particular replay state to decide whether the event is admitted.

**THEOREM 2.2 (FORWARD PRESERVATION).** *If  $\Sigma \vdash \sigma$ , event  $e_i$  satisfies the typing rule, and  $\sigma \models g_i$ , then  $\Sigma \vdash \sigma[w_i]$ .*

**PROOF.** Every written coordinate receives a declared domain member, and every unwritten coordinate retains one. Consider an arbitrary incidence  $(e, p)$ . If neither coordinate is written, its implication in (1) is unchanged. If at least one is written, rule (3) applies. Its first disjunct makes the post-event edge zero, either by an explicit write or by an unchanged singleton observation. The incidence implication is then vacuous. Its second disjunct makes the post-event endpoint one by the corresponding reasoning, so the implication's consequent holds. Both endpoint incidences hold for every edge, establishing the result.  $\square$

Simultaneous overwrite matters. Deleting an edge and one endpoint in one event is legal when the edge is set to zero; there is no intermediate state in which a still-present edge dangles. Conversely, setting an edge to one while leaving an endpoint unchanged requires a singleton observation that this endpoint is one. Deleting a node without deleting its incident edges can be admitted only when the unchanged edge-presence cells are observed zero. Thus creation and deletion are not treated as untyped Boolean writes even though their data representation is Boolean.

The rule is sufficient rather than complete. An unchanged edge observed present already implies, in every well-typed input, that both endpoints are present. The rule does not exploit that implication when another incidence changes. Rejecting some safe events is an admission boundary; it does not weaken the preservation theorem for admitted events. A richer type analysis could admit more inputs, but would require its own proof and could change the exact retention language if it introduced new dynamic conditions.

## 2.4 Complement receipts and their decoder

For a written cell  $x \in W_i$ , let its effective pre-value set be

$$A_i(x) = \begin{cases} g_i(x), & x \in R_i, \\ D_x, & x \notin R_i. \end{cases} \quad U_i = \{x \in W_i \mid |A_i(x)| > 1\}. \quad (4)$$

A receipt for an executed occurrence is  $h_i = (i, \eta_i)$ , where  $i$  identifies the original occurrence and  $\eta_i$  is the map  $\sigma_i \upharpoonright U_i$  of actual old values. Cells in  $W_i \setminus U_i$  need no stored coordinate because their singleton  $A_i(x)$  determines the old value. Unwritten cells need no stored coordinate because the event does not destroy them. Receipt domains are exact, not optional supersets.

Given post-state  $\tau$ , the inverse associated with  $e_i, h_i$  first checks  $\tau(x) = w_i(x)$  for every  $x \in W_i$ . It restores each written cell from  $\eta_i(x)$  when  $x \in U_i$ , and from the unique member of  $A_i(x)$  otherwise. It leaves unwritten cells unchanged and checks all observations in the restored state. These checks make the inverse a partial operation. An arbitrary supplied receipt need not be valid, and an inverse presented in the wrong order need not be enabled.

**THEOREM 2.3 (ONE-STEP ROUND TRIP).** *If  $\sigma \xrightarrow{e_i} \tau$  and  $h_i$  is generated from this step by the receipt rule, then the inverse of  $e_i, h_i$  applied to  $\tau$  is defined and returns exactly  $\sigma$ .*

**PROOF.** On a written cell,  $\tau$  has the required post-value. If  $A_i(x)$  is a singleton, enabledness implies that  $\sigma(x)$  is its unique member. Otherwise the receipt contains  $\sigma(x)$  by construction. The inverse therefore restores every written coordinate correctly. An unwritten coordinate already agrees with  $\sigma$  and is unchanged by the inverse. Cellwise equality follows, and the final observation check succeeds because  $\sigma$  enabled the original event.  $\square$

**COROLLARY 2.4 (TRACE ROUND TRIP AND BACKWARD PRESERVATION).** *Reversing the receipts of an admitted execution in reverse occurrence order recovers its base. Every reconstructed intermediate state is well typed.*

**PROOF.** The last receipt sees its actual forward post-state. Theorem 2.3 recovers its pre-state, which is the preceding receipt's post-state. Induct backward until the base is reached. Each reconstructed state is an actual forward intermediate state, so it is well typed by Theorem 2.2 and initial typing.  $\square$

Backward preservation here follows from validated correspondence to a forward execution. It does not assert that any locally plausible inverse supplied by an untrusted producer preserves types. This asymmetry is intentional: the checker can replay the short admitted trace and compare every old value instead of trusting a producer's explanation of where that value came from.

The design resembles established round-trip and complement interfaces in bidirectional programming [Foster et al. 2007; Hofmann et al. 2011]. Its additional policy problem concerns retaining some captured events, which can change the predecessor against which a complement must be interpreted. A receipt is therefore associated with both an occurrence and a particular valid replay, not merely with the source-level name of an aspect.

## 2.5 A qualified necessity theorem for logged coordinates

A global claim of minimum trace information would depend on an encoding model, state constraints, and which queries must be answered. We make a smaller precise claim. Consider the scalar product fragment with no relational graph constraints. The decoder knows the event and post-state, and the old-value representation stores only a chosen set of overwritten coordinates as coordinates. It may not encode several old coordinates into a new auxiliary value.

**THEOREM 2.5 (COORDINATE NECESSITY IN THE PRODUCT FRAGMENT).** *To invert an event for every enabled product-state input in this storage model, every written  $x$  with  $|A_i(x)| > 1$  must be stored. Storing exactly these coordinates is sufficient.*

**PROOF.** Sufficiency is Theorem 2.3. For necessity, suppose a written coordinate  $x$  with at least two allowed old values is omitted. Choose two distinct members of  $A_i(x)$  and two input states differing only at  $x$ . Fix every other observed coordinate to an allowed value and every other coordinate to any domain member. The product assumption makes both states available and enabled. Their post-states are identical because  $x$  is overwritten by the same constant. Their stored old coordinates are also identical because  $x$  was omitted. A decoder receiving the same event, post-state, and stored data cannot return two different pre-states. This contradicts inversion for all enabled inputs.  $\square$

For fixed unwritten coordinates the event's preimage fiber has size  $\prod_{x \in W_i} |A_i(x)|$ . An arbitrary lossless fixed-width binary description of a member of that fiber needs at least

$$\left\lceil \log_2 \prod_{x \in W_i} |A_i(x)| \right\rceil \quad (5)$$

bits in the worst case, by injectivity of decoding. Coordinate storage is not claimed to attain this bound. For example, three old coordinates of domain size three can be jointly encoded in five bits, whereas three separate two-bit fields use six. This example concerns representation, not a failure of round trips.

Graph constraints can make a coordinate inferable as well. If an unchanged edge is present, its endpoint values are determined by typing. Storing one endpoint's old presence may then be unnecessary even when its unary observation alone is not a singleton. Theorem 2.5 deliberately excludes that relational inference. No globally minimum typed-graph log or minimum encoding of all future retention queries follows from it.

## 2.6 Selective replay changes the complement context

**THEOREM 2.6 (REBASED SELECTIVE RECEIPTS).** *For a valid retention  $K$ , generating new receipts while executing  $T \upharpoonright K$  yields a trace round trip to  $B$ . Reusing the original receipts of the retained events does not, in general, have this property.*

**PROOF.** The positive claim applies Theorem 2.3 and Corollary 2.4 to the selected execution. For the negative claim, take one cell of domain  $\{0, 1, 2\}$  initially zero. Event  $a$  writes one and event  $b$  writes two; neither has an observation. In the full trace,  $b$ 's receipt stores one. Retaining only  $b$  is valid and ends at two, but its actual pre-state is zero. Restoring its original receipt returns one, not the base. The unchanged final value two cannot detect the stale complement.  $\square$

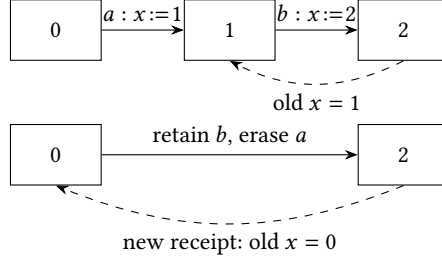


Fig. 1. The same retained overwrite has a different predecessor after erasure. Solid arrows are forward execution; dashed arrows are receipt-backed reversal. The original receipt for  $b$  is valid only on the upper execution.

The counterexample separates three checks that are sometimes conflated. The selected mask can be executable. Its claimed final model can be correct. Its claimed inverse can still be wrong. The outcome certificate used later validates all three, including equality of every stored old value with the pre-state of the selected execution. Merely checking that the inverse's postcondition matches the final write of two would not be enough.

Rebasing does not require recovering an arbitrary past model from a damaged final state. The artifact's semantic input includes  $B$  and the ordered event instances. It recomputes the selected predecessor and emits a new receipt. A system that retains only the final state and an old log would need a separate reconstruction strategy; our theorem does not silently give that different interface the same capabilities.

## 2.7 Uniform limits of reduced trace interfaces

The positive round-trip result requires a specified event interface. To see why an interface cannot in general be reduced to writes, snapshots, or unordered receipts, consider what a decision procedure must distinguish. Let an abstraction  $\alpha(B, T)$  be all information exposed to such a procedure. If two admitted inputs have the same abstraction but give different validity answers to the same named retention, no procedure using only that abstraction and retention can be correct on both inputs. This is an indistinguishability argument, not a lower bound on the runtime of a particular implementation.

**THEOREM 2.7 (OBSERVATIONS CANNOT UNIFORMLY BE OMITTED).** *The base, named writes, ordered original intermediate states, original complements, and original final state do not suffice to decide every observation-stable retention when the observation specifications are omitted.*

**PROOF.** Use one Boolean cell initially zero. The ordered events are  $a$ , writing one;  $b$ , writing zero; and  $r$ , which performs no write. In the first input  $r$  observes  $\{0\}$ ; in the second it observes  $\{0, 1\}$ . Both full traces execute. Their original states are 0, 1, 0, 0 in order. The writer receipts are zero for  $a$  and one for  $b$ ; the reader has an empty receipt. Thus all listed exposed information agrees. Retain  $a$  and  $r$ , and erase  $b$ . Immediately before the reader the cell is one. The first specification rejects and the second accepts. A decision procedure receiving the common exposed information and common retention therefore cannot return the required answer on both inputs.  $\square$

The theorem does not require that an event's observation be equality. Indeed, the ability to distinguish a consumed value from a wider acceptable test is what makes the example informative. Adding all original state snapshots does not repair this loss: a snapshot records which value

occurred, not which alternatives would have met the captured contract. Recording an accepting set is consequently specification information rather than a redundant copy of the observed state.

**THEOREM 2.8 (UNORDERED RECEIPTS DO NOT REPLACE OCCURRENCE ORDER).** *The base, final state, named writes, per-occurrence original old values, and observations do not suffice uniformly when the relative order of the named occurrences is omitted.*

**PROOF.** Start a Boolean cell at zero and use blind writes  $a : 1, b : 0, d : 1$ , and  $e : 0$ , followed by a reader  $r$  accepting only zero. Compare

$$a b d e r \quad \text{and} \quad d e a b r.$$

Both full executions are valid and finish at zero. The old value associated with  $a$  is zero in both, with  $b$  it is one, with  $d$  it is zero, and with  $e$  it is one. The named writes and the reader's observation are also identical. Retain exactly  $a, e, r$ . The first selected execution writes one and then zero before  $r$ , so it succeeds. The second writes zero and then one before  $r$ , so it fails. Once occurrence order is removed, the specified records cannot distinguish the two required answers.  $\square$

This counterexample keeps original old values *attached to their named occurrences*; it does not rely on confusing two unlabelled receipts. It shows that the old value observed on one execution does not in general determine where that occurrence belongs relative to a different retained subset. An exact representation may compress order, for example into the observation-specific representatives developed next, but cannot discard its relevant effect outright.

**Example 2.9** (An unchanged original snapshot may hide a necessary write). Let  $x$  initially be zero, and let both  $a$  and  $b$  blindly write one, in that order. The original transition performed by  $b$  changes no value, so its before/after snapshot difference is empty. Nevertheless, retaining  $b$  and erasing  $a$  must produce one. Replaying an empty difference for  $b$  instead produces zero. Adding a final reader accepting one makes the same loss change executable-retention answers: the captured overwrite still enables the reader, whereas the empty-difference replay does not. A delta that drops idempotent writes is therefore not an equivalent event representation for this semantics.

These examples establish three distinct uniform failure modes. Observation omission loses acceptable alternatives, order omission loses the relevant last writer, and snapshot-difference omission loses the write itself when its original execution happened to be idempotent. They do not establish that every field in every particular input is necessary, that the full input is globally minimum in bits, or that a richer delta calculus cannot preserve the required information. The coordinate lower bound in Theorem 2.5 and the local normal form in the next section have their own explicitly narrower comparison classes.

### 3 Exact Selective Replay by Last-Occurrence Normalization

Round trips validate a particular retained execution. Policy search additionally needs a characterization of *which* retained executions are possible. We first factor validity into conditions on individual observations. We then show that the effect of aspect grouping can be eliminated locally without enumerating all retention masks. The result is an exact Boolean formula whose clauses retain an operational meaning.

#### 3.1 Ghost prefixes are mathematical observations, not executions

For each position  $i$  and retention  $K$ , define  $G_i(K)$  by starting at  $B$  and applying the overwrites of retained events at positions smaller than  $i$ , ignoring all guards. Equivalently,  $G_i(K)(x)$  is the constant of the last retained earlier write to  $x$ , or  $B(x)$  if no such write exists. The prefix is defined

even when a retained event would have been disabled. It is not an alternative execution semantics, and it need not be well typed when guards are ignored.

For an observation  $o = (i, x, S)$ , with  $S = g_i(x)$  and reader tag  $r = a_i$ , put

$$H_o(K) \iff r \notin K \vee G_i(K)(x) \in S. \quad (6)$$

Skipping the reader makes its observation irrelevant. Including the reader asks only about the value its preceding retained writes would supply. A later retained event may have a failed ghost observation even if an earlier event already fails in actual replay; that fact remains a valid obstruction to success, but is not necessarily the first operational error.

**THEOREM 3.1 (EXACT OBSERVATION FACTORIZATION).** *For every retention  $K$ ,*

$$\text{Valid}_{B,T}(K) \iff \bigwedge_{i=0}^{n-1} \bigwedge_{x \in R_i} H_{(i,x,g_i(x))}(K). \quad (7)$$

**PROOF.** Suppose guarded replay succeeds. Before every retained event it has applied exactly the preceding retained overwrites, so its state equals  $G_i(K)$ . Each recorded observation therefore holds. A skipped reader satisfies its  $H_o$  by definition.

Conversely, suppose every  $H_o$  holds. Induct over original event positions. Initially actual replay and the ghost prefix both equal  $B$ . A skipped event changes neither. At a retained event, the induction hypothesis gives the same pre-state, and every observation is true by the corresponding  $H_o$ . The actual event is therefore enabled and both computations apply the same overwrite. Induction reaches the end and proves executability. Theorem 2.2 separately gives well-typedness of these actual intermediate states.  $\square$

The reverse implication is the reason ghost conditions are safe to use in a checker. A failed ghost condition excludes a successful guarded replay even if its prefix itself contains disabled events. No certificate must pretend that such a ghost prefix was an observed execution. Conversely, satisfying every condition repairs this apparent circularity by the induction above: the entire prefix then becomes a real execution.

The factorization would fail for captured updates with implicit reads. If a write recomputed a value from the current state, deleting an earlier event could change its constant, so the ghost last-writer calculation would no longer describe actual replay. The closed overwrite syntax and explicit observation discipline are therefore proof assumptions, not implementation conveniences.

### 3.2 A forced anchor for the reader's own tag

Fix  $o = (i, x, S)$  and condition on retaining its reader tag  $r$ . Every earlier occurrence tagged  $r$  is then retained as well. If any such occurrence writes  $x$ , let  $\alpha$  be the last one and set  $b = w_\alpha(x)$ . Otherwise take  $\alpha = -1$  and  $b = B(x)$ . We call  $(\alpha, b)$  the anchor. No write at or before  $\alpha$  can determine the observed value after the anchor's forced write.

Among events strictly between  $\alpha$  and  $i$ , retain only the last write to  $x$  of each other tag. Let the resulting ordered representatives be

$$(a_1, v_1), \dots, (a_\ell, v_\ell), \quad a_j \neq r, \quad (8)$$

where tags are pairwise distinct and the order is their original occurrence order. A representative is *good* when  $v_j \in S$  and *bad* otherwise. The words good and bad are relative to this one observation. The same tag may have a different role for another observation or another cell.

**LEMMA 3.2 (LAST-OCCURRENCE NORMALIZATION).** *Conditional on  $r \in K$ , the value  $G_i(K)(x)$  equals the result of starting at  $b$  and applying, in order, exactly the representatives whose tags belong to  $K$ .*

PROOF. The anchor, if an event, is forced retained. Writes at or before it cannot affect which write is last before the observation. Consider two writes after the anchor with the same tag. They are jointly retained or jointly absent. If retained, the later one supersedes the earlier one before position  $i$ ; if absent, neither matters. Removing all non-last writes of each tag leaves the last retained write unchanged. After this removal the surviving writes are precisely the ordered representatives. If none is retained, the anchor or base supplies  $b$ .  $\square$

Repeated tags are more than duplicate syntax. Before normalization, one tag can supply both acceptable and unacceptable values at different times. Treating its occurrences as independent retention variables is unsound; treating all its writes as equally relevant can obscure the local polarity structure. The lemma uses joint retention to show that only its last relevant occurrence can win. The independent checker later uses the raw trace rather than repeating this argument algorithmically.

For example, take base zero, two writes of tag  $a$  with values one and zero, then a write of tag  $g$  with value one, followed by  $r$  observing one. The full trace is valid. Keeping  $a$  retains both its writes, so  $a$  never supplies the last acceptable value when  $g$  is absent. The condition is simply that  $r$  requires  $g$ . The earlier good write of  $a$  cannot be used as an alternative provider. This example will also distinguish normalized local clauses from raw last-writer cuts in Section 6.

### 3.3 Constructing the local clause family

Let  $X_a$  mean that tag  $a$  is retained. Write  $P$  for all good representative tags and

$$P_{>j} = \{a_t \mid t > j, v_t \in S\}$$

for the good suffix after representative  $j$ . The preliminary clauses are

$$C_0 = \neg X_r \vee \bigvee_{a \in P} X_a \quad \text{if } b \notin S, \quad (9)$$

$$C_j = \neg X_r \vee \neg X_{a_j} \vee \bigvee_{a \in P_{>j}} X_a \quad \text{for each } v_j \notin S. \quad (10)$$

An empty positive disjunction is false. The empty conjunction of clauses is true. If  $C_0$  exists, delete  $C_j$  precisely when there is no good representative before  $j$ . Denote the remaining family by  $C_o$  and its conjunction by  $F_o$ .

The clauses have a direct reading. An unacceptable anchor needs a retained good writer whenever the reader is kept. A retained bad writer also needs a later retained good writer whenever the reader is kept. Later bad writers do not repair the earlier one; they create their own obligation. A skipped bad writer satisfies its own clause through its negative literal. The deletion rule removes a bad-writer obligation already entailed by the bad-anchor obligation.

**THEOREM 3.3 (EXACT LOCAL CLAUSES).** *The formula  $F_o$  denotes exactly  $H_o$  for every retention assignment.*

PROOF. If the reader is absent, every clause is true. Otherwise use Lemma 3.2. When the last retained representative is good, it satisfies  $C_0$  when present. Every retained bad representative is earlier than that last good representative, so its good suffix includes a true literal. Absent bad representatives satisfy their clauses negatively. If there is no retained representative and the anchor is good, there is no base clause and every bad representative is absent. These are exactly the ways the observation can hold.

For the remaining cases, if the anchor is bad and no representative is retained,  $C_0$  is false. If the last retained representative is bad, its clause is false: the reader and that writer are retained, but no

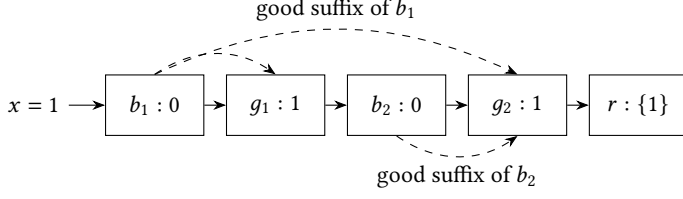


Fig. 2. A normalized profile for an observation of one. Writers are ordered left to right; the anchor is acceptable. Dashed links indicate the later good writers that can satisfy each bad writer's clause, not new event-order constraints.

later good writer is retained. Thus failure falsifies a preliminary clause. Deleting a clause subsumed by  $C_0$  does not change the conjunction, proving exactness for  $F_0$ .  $\square$

In an admitted full trace, every surviving clause has a positive literal. An all-negative clause would be false when all tags are retained, contradicting admission and Theorem 3.3. This is a derived property of valid input, not a reason for the compiler to discard an impossible clause in a malformed input. The checker first validates full-trace execution.

### 3.4 A worked profile with both replacement and inhibition

Let the anchor be good for the observation  $x = 1$ . In order, let the distinct tags  $b_1, g_1, b_2, g_2$  write 0, 1, 0, 1, and let  $r$  observe one. The normalized local formula is

$$(\neg r \vee \neg b_1 \vee g_1 \vee g_2) \wedge (\neg r \vee \neg b_2 \vee g_2), \quad (11)$$

where tag names abbreviate their Boolean variables. Retaining  $b_1, r$  and removing both good writers fails even if  $b_2$  is removed. Retaining  $g_1$  repairs that failure only when no later retained bad writer supersedes it. Retaining  $b_2$  therefore adds a separate need for  $g_2$ .

This profile cannot be summarized by saying that  $r$  requires any good writer. With  $g_1, b_2, r$  retained and  $g_2$  removed, a good writer exists but is too early. Nor can it be summarized by saying that every bad writer must be removed: either bad writer may remain when an appropriate later good writer repairs the value. The temporal suffix in (10) is the information that distinguishes these cases.

The same construction handles accepting sets without changing its proof. Replacing zero and one by any two values outside and inside  $S$  leaves the Boolean profile unchanged. It is membership, not a numerical distance from the threshold, that matters. Consequently, a local explanation can be identical for two quite different scalar domains while the captured event constants remain different.

### 3.5 An output-sensitive compilation procedure

One implementation scans the prefix for a selected observation to locate its last reader-tag write, then scans backward from the reader to the anchor, keeping the first encountered write of each other tag. Reversing that list produces the representatives. A backward pass accumulates the good suffix, and a forward bit records whether a good writer has occurred before each bad representative. These data suffice to emit (9)–(10) and apply the single subsumption test.

---

Phase Invariant after the phase

---

- 1 The anchor is the last forced write of the reader's tag, or the base.
  - 2 Every representative is the last post-anchor write of its tag; their order is original occurrence order.
  - 3 The suffix at position  $j$  contains exactly the good tags strictly later than  $j$ .
  - 4 An emitted bad clause has a good predecessor whenever a bad-base clause would otherwise subsume it.
- 

For a prefix of length  $i$  and  $m = |\mathcal{A}|$  tags, identifying representatives takes  $O(i + m)$  elementary indexing work with a last-write table. Emitting explicit clauses takes time proportional to their literal output, at most  $O(m^2)$  per observation. A naive all-observation implementation can rescan prefixes and cost  $O(qn + qm^2)$ , where  $q$  is the number of unary observations. The artifact uses a bounded straightforward implementation rather than claiming a new optimal global compilation algorithm.

The formulas may share many suffixes. A compressed decision representation can avoid copying them; the next section's quadratic lower bound is deliberately about *flat* CNF over original tag variables. Compilation is exact regardless of whether a later solver shares data internally. The distinction prevents a bound on printed literals from being mistaken for a lower bound on all semantic representations or all query algorithms.

#### 4 Local Prime Clauses and Minimum Failure Explanations

The local formula is exact, but exactness alone does not establish minimality. This section specifies the representation and quantifiers under which a minimum claim is valid. Its Boolean foundation is standard prime-implicate reasoning; the source of tractability is that normalization makes each observation unate. The distinction becomes important as soon as several observations are combined.

##### 4.1 Prime implicates and unate functions

A non-tautological clause  $C$  is an implicate of a Boolean function  $H$  when  $H \models C$ . It is prime when no proper subclause remains an implicate. Clauses are regarded as sets of distinct literals. A CNF is *unate* when a variable never occurs positively in one clause and negatively in another. Renaming some variables to their complements makes a unate CNF positive, and hence its function monotone increasing in the renamed variables. This monotonicity is not monotonicity in the original retention decisions.

For the normalized observation formula, the reader and bad representative tags are negative; good representative tags are positive. Last-occurrence normalization is what gives each other tag exactly one of these roles. A tag can still have opposite roles in different observation formulas, so their global conjunction need not be unate.

LEMMA 4.1 (THE UNATE ANTICHAIN PRINCIPLE). *The distinct non-subsumed clauses of a unate CNF are exactly all prime implicates of its function.*

PROOF. Complement selected variables so every input clause is positive. Let the resulting monotone function be  $H$ . A prime implicate cannot contain  $\neg z$ . Otherwise, since removing that literal destroys implication, there is an assignment satisfying  $H$  and falsifying the other literals with  $z = 0$ . Increasing  $z$  to one preserves  $H$  by monotonicity and falsifies the whole original clause, a contradiction. Thus all prime implicates are positive.

For a positive implicate  $D$ , set all variables of  $D$  to zero and all other variables to one. Because  $D$  is false,  $H$  is false. Some input clause  $C$  is false, so every variable of  $C$  lies in  $D$ , giving  $C \subseteq D$ . An input clause not subsumed by another cannot have a proper implicate subclause: that subclause

would contain an input clause strictly smaller than the original. Conversely, a subsumed clause cannot be prime. Complementing the renamed variables back preserves implication and literal containment.  $\square$

The role of essential prime clauses in representation size is established in knowledge compilation [Darwiche and Marquis 2002]. We include the elementary argument to make clear which part is being used. It does not depend on a model-weaving interpretation, and it is not presented as a new Boolean theorem.

#### 4.2 Why the normalized clauses are the entire local prime set

**THEOREM 4.2 (CANONICAL LOCAL PRIME REPRESENTATION).** *For each observation  $o$ ,  $C_o$  is exactly the set of all prime implicates of  $H_o$ .*

**PROOF.** Theorem 3.3 establishes equivalence, and the representative tags make  $F_o$  unate. Distinct bad-writer clauses cannot subsume one another: each has a negative bad-tag literal absent from the other. A bad-writer clause cannot subsume the base clause because the base clause lacks that negative bad tag. The base clause subsumes a bad-writer clause exactly when all good tags occur in that writer's good suffix, equivalently when no good representative precedes it. The construction deletes exactly those clauses. All remaining clauses are therefore distinct and non-subsumed. Apply Lemma 4.1.  $\square$

This proof addresses both directions needed for an explanation claim. Every emitted clause is necessary for the local function; no clause outside the family can offer a shorter prime explanation. Checking only that emitted clauses are implicates would establish sound explanations, but not that a smaller one was not missed. Checking only all subset deletions of one emitted clause would establish inclusion-minimality, but not minimum cardinality among different clauses.

The construction need not enumerate all  $3^m$  signed clause candidates to obtain this completeness. It uses the semantic profile and the structural unateness lemma. The small oracle deliberately takes the expensive alternative over four tags: it enumerates every potential signed clause and checks implication directly. This offers a bounded implementation test whose logic is not merely a duplicate of the construction.

#### 4.3 A representation-qualified optimality result

A flat CNF here uses only the original aspect variables. It has no auxiliary variables, shared suffix nodes, existential quantification, or alternate circuit representation. Clause count counts non-tautological clauses, and literal count counts occurrences of signed variables across clauses.

**COROLLARY 4.3 (MINIMUM FLAT CNF).** *Among flat CNFs over original aspect variables that denote  $H_o$ , the normalized CNF has minimum clause count and minimum total literal count. A literal-minimum CNF is unique up to clause and literal order.*

**PROOF.** Every non-tautological clause in an equivalent CNF is an implicate of  $H_o$  and contains some prime implicate. Replace each clause by a contained prime. The replacement does not increase either count. It preserves equivalence because  $H_o$  implies each replacement, while each replacement implies the old clause. The resulting CNF consists of local primes. Omitting any prime from the non-subsumed unate family changes its function by Lemma 4.1; thus every normalized prime must occur. This yields both lower bounds. Equality in total literal count permits neither an extra nonempty clause, a proper superclause, nor a duplicate. The constant-true function has the unique empty CNF.  $\square$

Clause-minimum representations alone need not be unique in arbitrary Boolean languages, and the statement above does not rely on such a general claim. The stronger uniqueness concerns total literals in this specific unate family. Likewise, a circuit with auxiliary state can share a good suffix among several bad-writer obligations. Such a circuit may be smaller without contradicting the flat-CNF result.

#### 4.4 From a falsified prime to minimum forcing facts

Fix a full retention mask  $M$  for which  $H_o(M)$  is false. A local explanation is a pair of disjoint sets  $(P, D)$  with  $P \subseteq M$  and  $D \cap M = \emptyset$ . It records facts consistent with the queried mask: keep every tag in  $P$ , remove every tag in  $D$ , and leave the rest unspecified. It *forces local failure* when

$$\forall K \subseteq \mathcal{A}. \quad (P \subseteq K \wedge K \cap D = \emptyset) \implies \neg H_o(K). \quad (12)$$

Its size is  $|P| + |D|$ . Importantly, the quantifier ranges over every completion, not only completions valid at other observations.

**COROLLARY 4.4 (CARDINALITY-MINIMUM LOCAL OBSTRUCTION).** *A smallest local forcing explanation consistent with  $M$  is obtained by choosing a shortest clause in  $C_o$  falsified by  $M$ , and negating its literals to obtain keep/remove facts.*

**PROOF.** Condition (12) is equivalent to

$$H_o \models \left( \bigvee_{a \in P} \neg X_a \right) \vee \left( \bigvee_{a \in D} X_a \right).$$

Thus the negated facts form an implicate of  $H_o$ . Every implicate contains a prime implicate. By Theorem 4.2, all such primes are in  $C_o$ . A contained prime uses a subset of the original facts, remains falsified by  $M$ , and is no longer. Conversely each falsified prime supplies forcing facts. Choosing the shortest therefore attains minimum cardinality.  $\square$

This is the familiar relationship between compatible prime explanations and Boolean decisions, specialized to the failure of one observation [Darwiche and Hirth 2023]. The algorithmic benefit is that the relevant prime set is available directly from the trace profile. General explanation algorithms address richer function representations and need not inherit this structure [Darwiche and Ji 2022]. We do not use the word “minimal” to replace the cardinality quantifier: removing any fact from one explanation is a weaker test than comparing it to every other explanation.

The witness concerns the *facts*, not a minimum residual subgraph. A certificate may name one cell and observation, but the optimized object is a set of aspect-retention decisions. There may be several tied shortest clauses; the producer may choose any of them. No canonical tie-breaking theorem is needed for their semantic validity, although the artifact uses deterministic ordering for reproducibility.

#### 4.5 A matching size family and its interpretation

**THEOREM 4.5 (LOCAL OUTPUT AND EXPLANATION SIZE).** *For  $m$  tags, one observation has  $O(m)$  local prime clauses and  $O(m^2)$  literal occurrences. The quadratic literal bound and a linear minimum local explanation length are attained up to constant factors in the flat-CNF representation.*

**PROOF.** There are at most  $m - 1$  representatives, one clause per bad representative, and at most one base clause. Each clause uses at most  $m$  literals. For a matching family, use a good anchor and the ordered writers

$$b_1, g_1, b_2, g_2, \dots, b_k, g_k,$$

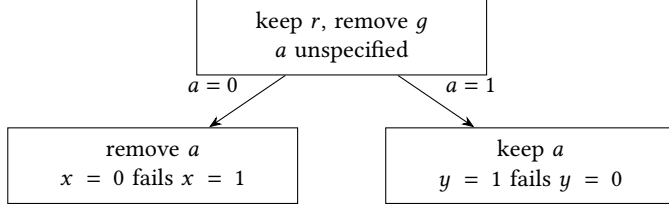


Fig. 3. Local failure has a fixed observation; global failure may use different observations in different branches. Under keep  $r$  and remove  $g$ , either decision for  $a$  prevents a valid replay, although neither local condition alone has a two-fact forcing explanation.

followed by reader  $r$ , where  $b_j$  writes a bad value and  $g_j$  a good value. There are  $2k + 1$  tags. Its  $k$  prime clauses have lengths  $k - j + 3$ , so their total is

$$\sum_{j=1}^k (k - j + 3) = 2k + \frac{k(k+1)}{2}.$$

Corollary 4.3 makes this total unavoidable for an equivalent flat CNF. Retain only  $b_1$  and  $r$ . The only falsified prime is the first, whose length is  $k + 2$ . Corollary 4.4 gives a shortest local explanation of that length.  $\square$

A last-writer decision list for this same observation is linear: inspect retained representatives from latest to earliest and return the first value's membership, falling back to the anchor. Therefore the quadratic literal result is not a quadratic information lower bound. It is the cost of making every local prime explicit without sharing. Similarly, a long shortest explanation need not make deciding this one observation difficult. Evaluating a fixed mask remains a scan.

Decision lists have their own rigorous explanation literature [Ignatiev and Marques-Silva 2021]. Our local list tests each representative tag only once after normalization, with a distinguished reader controlling relevance. The article does not infer that all decision lists have the same easily enumerated prime family. Repeated conditions, alternate tests, or a different global combination can remove the unate structure used here.

#### 4.6 A two-observation counterexample to global minimality

Use cells  $(x, y)$  initially  $(0, 0)$ . Aspect  $a$  writes  $(1, 1)$ , aspect  $g$  then writes  $(1, 0)$ , and aspect  $r$  observes  $x = 1$  and  $y = 0$ . The local formulas are

$$H_x = (\neg r \vee a \vee g), \quad H_y = (\neg r \vee \neg a \vee g). \quad (13)$$

Their conjunction simplifies to

$$H_x \wedge H_y \equiv \neg r \vee g. \quad (14)$$

When  $r$  is kept and  $g$  removed, removing  $a$  makes the first observation fail; retaining  $a$  makes the second fail. Neither choice can save the whole replay.

For the full mask  $M = \{r\}$ , the first observation's minimum forcing facts are keep  $r$ , remove  $a$ , remove  $g$ : three facts. But the global policy keep  $r$ , remove  $g$  already has no valid completion: two facts. This policy is minimum because the full trace witnesses every one-fact keep request and the empty trace witnesses every one-fact remove request. The shorter global explanation does not force one fixed observation to fail under every completion; it forces at least one of two observations to fail, depending on  $a$ .

This example also shows why a local syntactic rejection is not a global impossibility theorem. The two local clauses are ternary, yet the whole function has a binary implication representation. Concluding that no dependency graph exists merely because one local clause is not binary would be incorrect. Section 5 gives the exact semantic criterion and separately a sufficient, conservative syntactic fragment.

A different issue arises with raw failure cuts. In the repeated-tag example  $a : 1, a : 0, g : 1, r : \{1\}$  from Section 3, the normalized prime is  $\neg r \vee g$ . A raw bad-base cut requires  $a$  absent because the earlier  $a : 1$  is a potential good write. A raw bad-write cut at  $a : 0$  requires  $a$  present. Together the two cases explain why  $a$  is irrelevant, but neither single cut expresses the two-fact local prime. The checker therefore does not validate local minimum cardinality by trusting a single raw cut. It uses an independent finite truth-table calculation, while global refutations may branch between cuts.

## 5 Expressivity, Dependency, and the Two-Write Boundary

A local observation is easy to normalize. A policy, however, constrains the conjunction of all observations, with the same aspect variables shared across them. This section characterizes that conjunction and identifies exactly where an implication-closure account is sufficient. It then separates the number of writes per cell from the size of local explanations and from global search complexity.

### 5.1 The exact retention language without hidden aspects

Let  $\mathcal{M}_2$  be the class of Boolean functions expressible by conjunctions of non-tautological clauses, each containing one or two negative literals and at least one positive literal. The empty conjunction is permitted. Variables are the visible aspect tags, not auxiliary variables that are existentially projected after compilation. Introducing fresh scalar cells is different: cells store model state, while extra aspect variables would introduce new policy decisions.

**THEOREM 5.1 (EXACT SCALAR EXPRESSIVITY).** *The valid-retention functions of admitted scalar constant-overwrite traces with unary observations are exactly  $\mathcal{M}_2$ . Boolean cells suffice for the converse construction. The same  $\mathcal{M}_2$  upper bound holds for the declared graph-presence typing fragment.*

**PROOF.** For the upper bound, combine Theorems 3.1 and 3.3. Each local clause has the negative reader literal and at most one negative bad-writer literal. Its variables are distinct. It has at least one positive literal, because the admitted full retention satisfies it, whereas an all-negative clause would be false on that retention. Thus the global conjunction is in  $\mathcal{M}_2$ . For typed graphs, Theorem 2.2 ensures that satisfied observations along actual replay preserve typing; no additional mask condition is introduced.

For the converse, take an  $\mathcal{M}_2$  conjunction. Allocate one fresh Boolean cell initially zero for each clause. For a clause  $\neg r \vee p_1 \vee \dots \vee p_k$ , let the  $p_j$  tags each write one to that cell, in any fixed order, followed by an event of tag  $r$  observing one. It is executable exactly when  $r$  is absent or some  $p_j$  is retained. For a clause  $\neg r \vee \neg b \vee p_1 \vee \dots \vee p_k$ , first let tag  $b$  write one, then let the  $p_j$  tags write zero, then let  $r$  observe zero. It is executable exactly when  $r$  or  $b$  is absent, or some repairing  $p_j$  is retained.

Concatenate the gadgets. Their cells are fresh, so events of one gadget cannot change another gadget's observations. Shared tags enforce the same retention decision in every gadget. Factorization therefore gives exactly the conjunction. At full retention, each gadget's last positive writer establishes its observation, so the full trace is valid. There is one cell per clause and an event count linear in literal count. The empty conjunction is represented by an empty trace.  $\square$

The converse is not an implementation of arbitrary source aspects. It constructs a trace with a specified Boolean retention function. A tag may occur repeatedly across clause gadgets. The stronger hardness result below separately removes that repetition. The theorem also is not a classification of every possible reversible language; it identifies the functions generated by this particular captured-event interface.

A useful consequence is that the empty/full endpoint property does not characterize the language. There are Boolean functions accepting both endpoints that cannot be represented without additional aspect decisions.

COROLLARY 5.2 (A STRICT NO-HIDDEN-VARIABLE BOUNDARY). *The function*

$$F(a, b, c, d) = \neg a \vee \neg b \vee \neg c \vee d \quad (15)$$

*is true on empty and full retention but is not the valid-retention function of any admitted trace over exactly these visible variables.*

PROOF. Every non-tautological implicate of a single non-tautological clause contains that clause. Otherwise one can falsify the proposed implicate and make a missing literal of the original clause true, contradicting implication. Thus every non-tautological implicate of (15) contains at least three negative literals. If an equivalent  $M_2$  conjunction existed, each of its clauses would be an implicate with at most two negatives, which is impossible. The function is not constant true, so the empty conjunction is not an exception. Direct substitution verifies that the all-zero and all-one assignments both satisfy it.  $\square$

The boundary is about equivalence on visible masks. Adding an optional helper tag and asking whether some setting of it works changes the interface to existential projection. It can increase expressiveness, just as auxiliary variables change representation size. Our converse theorem and local size bounds explicitly avoid claiming that such extensions leave the language unchanged.

## 5.2 Exactly when implications describe the valid masks

A dependency graph with edge  $a \rightarrow b$  expresses the requirement that retaining  $a$  requires retaining  $b$ . Every family defined by such implications is closed under union and intersection. In a union, an antecedent occurs in at least one member and brings its consequent with it. In an intersection, it occurs in both members and its consequent occurs in both as well. This property gives a stronger test than asking whether validity is monotone under erasure.

Two examples separate the closure requirements. For base zero,  $a : 1, b : 1, r : \{1\}$ , the retentions  $\{a, r\}$  and  $\{b, r\}$  are valid, but their intersection  $\{r\}$  is not. Alternative writers defeat intersection closure. For base zero,  $a : 1, b : 0, r : \{0\}$ , the retentions  $\{r\}$  and  $\{a\}$  are valid, but their union  $\{a, r\}$  is not. Inhibition defeats union closure. Neither valid-mask family can be exactly a fixed conjunction of tag implications.

THEOREM 5.3 (EXACT IMPLICATION CRITERION). *Let  $\mathcal{V} \subseteq \mathcal{P}(\mathcal{A})$  be finite and contain both  $\emptyset$  and  $\mathcal{A}$ . It is definable by tag-to-tag implications if and only if it is closed under binary union and intersection.*

PROOF. Necessity follows from the preceding elementwise argument. For sufficiency, for each  $a \in \mathcal{A}$  define

$$\text{cl}(a) = \bigcap \{K \in \mathcal{V} \mid a \in K\}.$$

The set being intersected is nonempty because  $\mathcal{A} \in \mathcal{V}$ , and finiteness plus intersection closure gives  $\text{cl}(a) \in \mathcal{V}$ . Require  $a \rightarrow b$  for every  $b \in \text{cl}(a)$ . Every member of  $\mathcal{V}$  satisfies these implications.

Conversely, let  $K$  satisfy them. Each  $a \in K$  has  $a \in \text{cl}(a) \subseteq K$ , so  $K = \bigcup_{a \in K} \text{cl}(a)$ . A nonempty such union lies in  $\mathcal{V}$  by finite union closure. For  $K = \emptyset$ , use the explicit empty-set assumption. The implications therefore define exactly  $\mathcal{V}$ .  $\square$

The representation is generally a preorder rather than a partial order on the original tags. If two tags always occur together in valid masks, they imply one another. Collapsing mutual implication into strongly connected components gives a partial order on those components. This is not the event-order relation used for commutation: a chronological event order cannot contain a cycle, while a grouped-tag requirement relation can.

**COROLLARY 5.4** (A SYNTACTICALLY CHECKABLE IMPLICATION FRAGMENT). *If every nontrivial normalized local prime is binary, their conjunction is an exact implication representation of valid retention. If each tag occupies one contiguous trace block, the nontrivial requirements are acyclic.*

**PROOF.** Both empty and full retention satisfy every local clause. A nontrivial binary clause therefore has one negative and one positive literal, so is an implication. Theorems 3.1 and 3.3 give exactness of their conjunction. For a contiguous reader block, an earlier write of its own tag is forced when the reader is kept and makes the original observation unconditional if no other block intervenes. Otherwise a relevant writer lies in an earlier block. Every nontrivial requirement thus points to an earlier block and cannot form a cycle.  $\square$

The binary-local test is sufficient, not necessary. The two-observation example in (13) has two ternary local primes but simplifies globally to  $r \rightarrow g$ . A failed syntactic test must not be reported as a proof that a dependency representation is impossible. Such a proof needs a closure counterexample or another semantic argument.

### 5.3 The one-write fragment admits a unique maximum completion

A policy is a disjoint pair  $(P, D)$  of forced-kept and forced-removed tags. It is feasible when some  $K \in \mathcal{V}_{B,T}$  contains  $P$  and avoids  $D$ . Tags outside  $P \cup D$  are optional. With unit costs, collateral removal is

$$\text{cost}_{P,D}(K) = |(\mathcal{A} \setminus (P \cup D)) \setminus K|.$$

Requested removals do not count as collateral, and a forced retention cannot be removed to reduce cost.

A cell's write count is the number of event occurrences that write it, including writes that leave its original value unchanged. It is not the number of distinct constants, nor the number of distinct aspect tags. This matters because an idempotent write may become the only provider after another write is removed.

**THEOREM 5.5** (ONE WRITE PER CELL). *If each cell is written at most once in the entire trace, valid retentions are exactly a conjunction of binary requirements. Every feasible policy has a unique maximum valid completion, computable by closure propagation, which is also its unique minimum-unit-cost collateral-removal solution. Every infeasible consistent policy has a cardinality-minimum conflict of two facts.*

**PROOF.** Consider one observation. If it has no earlier writer, its base value is accepted because the full trace is valid. If its only earlier writer shares the reader's tag, retaining the reader forces that writer, so the observation is unconditional. Otherwise only two values are possible: the base and the unique writer's constant. Full-trace validity says that constant is accepted. If the base is accepted too, there is no condition. If not, the reader tag  $r$  requires the writer tag  $a$ . These are all observation conditions; factorization makes their conjunction exact.

Given  $(P, D)$ , start with  $Z = D$  and repeatedly add  $r$  to  $Z$  whenever  $r \rightarrow a$  and  $a \in Z$ . Every valid policy completion excludes  $Z$ , by induction over propagation: a reader requiring a removed writer must itself be absent. At the fixed point,  $K^* = \mathcal{A} \setminus Z$  satisfies every requirement, because a retained  $r$  with removed required  $a$  would have been added to  $Z$ . It avoids  $D$  and is a completion precisely when  $P \cap Z = \emptyset$ . Every possible completion is a subset of  $K^*$ , so it is the unique maximum. Any strict smaller completion removes an additional optional tag, giving strictly greater unit collateral cost.

If the policy is infeasible, some  $p \in P$  reaches some  $d \in D$  through the requirement relation. The pair keep  $p$ , remove  $d$  is already infeasible. No single keep fact conflicts, since full retention witnesses it; no single remove fact conflicts, since empty retention witnesses it. Two is therefore minimum. The policy's consistency excludes a contradictory keep/remove request on the same tag. After scanning the trace, reverse reachability from  $D$  computes  $Z$  and a conflict path in time linear in the requirement graph.  $\square$

For example, events  $a : x := 1$ ,  $b : \{x = 1\}; y := 1$ , and then another event of  $a$  observing  $y = 1$  use one write per cell. The requirements are  $b \rightarrow a$  and  $a \rightarrow b$ . Empty and full retention are valid, but neither singleton is. The tag cycle is not a temporal cycle: the first occurrence of  $a$  precedes  $b$ , which precedes the second occurrence of  $a$ . Closure on strongly connected components handles it without changing the theorem.

The unique-maximum property explains why unit-cost optimization adds no search in this fragment. Every optional tag not forced out can remain simultaneously. For zero-cost or non-unit objectives, uniqueness of a cost optimum would need a separate qualification, although the set-theoretic maximum remains the same. The implementation's simple repeated edge scans are polynomial; the linear-time claim concerns the standard graph-reachability algorithm, not an unmeasured micro-optimization of that implementation.

#### 5.4 Commutation is a separate, conservative guarantee

For events  $e_1, e_2$ , write  $R_j$  for observed cells and  $W_j$  for written cells. Declare them independent when

$$W_1 \cap (R_2 \cup W_2) = \emptyset \quad \text{and} \quad W_2 \cap (R_1 \cup W_1) = \emptyset. \quad (16)$$

The test excludes write/write and write/observation overlap. It can reject events that happen to commute on particular values, such as equal constant writes. Its purpose is a transparent sufficient condition with a receipt guarantee.

**THEOREM 5.6 (COMMUTING DIAMOND WITH COMPLEMENTS).** *If independent events execute consecutively from a well-typed state, they can execute in the opposite order. Both orders have the same final state and the same complement for each corresponding occurrence.*

**PROOF.** The first event does not change the second's observed cells. Therefore the second was already enabled before the first. Executing the second cannot change the first's observed cells, so the first remains enabled afterward. Their disjoint write sets imply equality of the final states cell by cell. A complement records only pre-values of written cells, and neither event writes the other's written cells. Thus the old value stored for either occurrence is independent of their order. Types are preserved on both routes by Theorem 2.2.  $\square$

Orient every non-independent event pair according to the original trace order and take transitive closure. This relation is acyclic because every edge goes forward in the original order. Any two linear extensions are connected by swaps of adjacent incomparable events: move the desired first element leftward across preceding elements, all of which must be incomparable with it, then induct

on the remaining suffix. Each swapped pair is independent by construction. Applying Theorem 5.6 along the swaps proves equal final states and corresponding receipts for all these reorderings of a valid retained trace.

This is trace-relative confluence within the declared independence relation. It is not confluence of unrestricted graph rewriting, nor an assertion that conflicting aspects can always be reordered. Axiomatic reversible-computation accounts distinguish several causal consistency, safety, and liveness properties; our footprint diamond is not claimed to instantiate all those axioms or replace that theory [Lanese et al. 2024]. It establishes exactly the swapping property needed for the stated occurrence-level scheduling freedom.

## 5.5 Why unrestricted finite policy search is hard

The preceding results might suggest that all policies reduce to closure plus a few choices among providers. Those choices can encode a satisfiability problem. Complexity is stated for unbounded finite inputs; the executable artifact’s admission limit of twelve tags is not an assumption of the theorem.

**THEOREM 5.7 (POLICY FEASIBILITY).** *Policy feasibility is NP-complete even for Boolean scalar cells, one event per aspect, exactly one forced-kept aspect, and exactly one forced-removed aspect.*

**PROOF.** Membership in NP follows by guessing  $K$  and directly replaying its events. For hardness, start with a 3-CNF formula with variables  $z_1, \dots, z_q$  and nonempty clauses  $C_1, \dots, C_h$ . Use initially-zero cells  $x_i, y_i$  for each variable and  $c_j$  for each clause. Introduce optional tags  $p_i, n_i$ , a reset  $d$ , and final reader  $r$ . Their single events occur in the order

$$p_1, \dots, p_q, d, n_1, \dots, n_q, r.$$

Event  $p_i$  has no observations, writes  $x_i = y_i = 1$ , and writes  $c_j = 1$  whenever positive literal  $z_i$  occurs in  $C_j$ . The reset writes every  $x_i = 0$ . Event  $n_i$  observes  $x_i = 0$ , writes  $y_i = 1$ , and writes  $c_j = 1$  whenever negative literal  $\neg z_i$  occurs in  $C_j$ . Reader  $r$  observes every  $y_i = 1$  and every  $c_j = 1$ .

The all-retained trace is valid: reset  $d$  enables every negative event, every  $y_i$  is written, and every nonempty clause has a literal writer. Force  $r$  kept and  $d$  removed. Retaining  $n_i$  now prohibits retaining  $p_i$  because of its  $x_i = 0$  observation. The final  $y_i = 1$  observation requires at least one of them. Thus exactly one is retained for every variable. Interpret  $p_i$  as true and  $n_i$  as false. A clause cell is one exactly when some selected literal satisfies that clause. Feasible completion is therefore equivalent to satisfiability. The construction is polynomial and uses one event per aspect.  $\square$

The reduction uses the classical NP-completeness of Boolean satisfiability, in the standard reduction framework [Karp 1972]. The new statement to inspect is the placement of that hardness inside this event language with this policy interface. It does not claim a new general SAT lower bound. In particular, the full original trace in the reduction is always valid, so hardness is not hidden in input admission.

The reset is important. Without it, retaining both  $p_i$  and  $n_i$  would fail in the full trace, violating the original-validity contract. With it, the full trace is a lawful input; removing the one designated reset exposes the exclusive choice. That mechanism retains the important non-monotone behavior rather than moving the hard problem into an invalid source trace.

## 5.6 Two writes per cell already suffice

In the preceding reduction, clause cells can have three literal writers. They can be replaced by a binary choice gadget while preserving one event per tag and the two-fact policy interface.

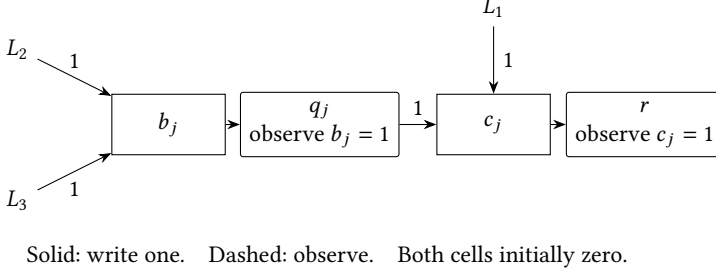


Fig. 4. The two-write clause gadget for  $L_1 \vee L_2 \vee L_3$ . An arrow from a literal tag denotes its write of one. The helper  $q_j$  observes the child cell before writing the root. The final reader observes the root. Each cell has at most two writer occurrences.

**THEOREM 5.8 (THE TWO-WRITE THRESHOLD).** *Policy feasibility is NP-complete when all cells are Boolean, each cell is written at most twice, each aspect has exactly one event, and the policy has exactly one forced retention and one forced removal. Every nontrivial local prime in this fragment has at most three literals.*

**PROOF.** Start with the reduction of Theorem 5.7, padding short nonempty clauses by repeated literals to length three. Each  $x_i$  has two writers,  $p_i$  and  $d$ , and each  $y_i$  has two,  $p_i$  and  $n_i$ . For clause  $L_1 \vee L_2 \vee L_3$ , replace its clause cell by initially-zero root  $c_j$  and child  $b_j$ . The tag for  $L_1$  writes  $c_j = 1$ . The tags for  $L_2$  and  $L_3$  write  $b_j = 1$ . Add optional tag  $q_j$  whose single event observes  $b_j = 1$  and writes  $c_j = 1$ . Place these  $q_j$  events after all variable events and before  $r$ , which observes every root  $c_j = 1$  as before. Repeated literals coalesce duplicate writes within one event; they do not create a third writer occurrence.

At full retention every child has a literal writer, every  $q_j$  runs, and every root is one. The policy still keeps  $r$  and removes  $d$ . The variable gadgets still force exactly one of  $p_i, n_i$ . If a completion is valid, a root is one because  $L_1$  is selected or  $q_j$  is retained. In the second case its observation requires  $L_2$  or  $L_3$  selected. Every original clause is therefore satisfied. Conversely, from a satisfying assignment keep its selected variable tags and keep  $q_j$  exactly when  $L_2$  or  $L_3$  is selected. Each retained  $q_j$  is enabled, and every root is one because the clause has a selected literal. This yields a valid completion. The construction is polynomial and direct replay gives NP membership.

For the local width bound, at most two earlier writer occurrences can survive normalization. A base clause contains the reader and at most two good tags. A bad-writer clause contains the reader, one bad tag, and at most one later good tag. An anchor can only remove representatives. Every local prime therefore has at most three literals.  $\square$

This is a sharp threshold in write count under the remaining declared conditions: at most one write gives closure by Theorem 5.5, whereas at most two already gives hardness. It is not a threshold for every other resource. An event may observe or write many cells, and the construction does not prove hardness with an additional constant bound on every event's footprint. It also does not require repeated occurrences of a tag, large domains, fresh allocation, or non-unary predicates.

**COROLLARY 5.9 (NO UNIVERSAL POLYNOMIAL REFUSAL GUARANTEE UNLESS NP=coNP).** *Unless NP = coNP, not every infeasible policy in the unbounded finite fragment has a polynomial-size refusal certificate checkable in polynomial time.*

**PROOF.** If all infeasible inputs had such certificates, the complement of the NP-complete feasibility problem would be in NP. This would imply NP = coNP.  $\square$

The statement concerns a simultaneous size and checking-time guarantee. A short certificate that a checker verifies by exhaustive exponential search does not contradict it. Neither does an exponential refutation tree with a simple local checker. The artifact deliberately uses bounded enumeration and finite branch proofs rather than claiming a compact complete proof system for all inputs.

### 5.7 Global conflicts can be large while local primes stay ternary

**THEOREM 5.10 (UNBOUNDED GLOBAL MINIMUM CORES).** *For every  $k \geq 2$ , there is an admitted Boolean trace with  $2k - 1$  aspects, one event per aspect, and two writes per cell, whose local prime clauses have size three but whose designated policy has minimum conflicting subpolicy size  $k + 1$ .*

**PROOF.** Take a full binary tree with  $k$  leaves. Allocate an initially-zero cell for each internal vertex. The aspect of each non-root vertex writes one to its parent's cell. An internal vertex's aspect also observes its own cell to be one. Order events from children to parents. Each internal cell has exactly two writers, one for each child, and every aspect occurs once. The full trace is valid. The local clause at internal vertex  $v$  is

$$\neg v \vee \text{left}(v) \vee \text{right}(v).$$

Force the root kept and all leaves removed. A retained internal vertex needs a retained child. Following retained children in a finite tree eventually reaches a retained leaf, contradicting the policy. Thus all  $k + 1$  facts conflict.

If the root-keep fact is deleted, empty retention is a witness. If the removal of one leaf is deleted, retain exactly that leaf and its path to the root. Every retained internal vertex then has a retained child, and no other leaf is kept. This is a valid witness. Every proper subpolicy is contained in one such single-fact deletion, and therefore is feasible. Hence  $k + 1$  is minimum cardinality, not merely the size of one inclusion-minimal candidate.  $\square$

The construction identifies the quantifier responsible for large cores. Each internal observation needs one of two children; the global policy forbids every root-to-leaf support path. A local clause mentions only one parent and two children, but the refusal must account for all leaf choices. No fixed bound on local clause width can bound the size of a global minimum policy obstruction. The finite tests instantiate two through six leaves; the general statement is supported by the argument above, not by extrapolating those five cases.

### 5.8 Collateral optimization has its own reduction

For completeness, minimum collateral deletion can be isolated from satisfiability. Take an undirected graph with ordered vertices. For each edge  $\{u, v\}$ ,  $u < v$ , allocate an initially-zero cell. Event  $v$  observes zero on its incoming edge cells, and event  $u$  writes one on its outgoing edge cells. Immediately after each vertex event, put a reset aspect that restores its outgoing cells to zero. The full trace is valid.

Force all resets removed and leave all vertex aspects optional. For an edge  $u < v$ , retaining both endpoints makes  $v$  observe one and fail. If no edge has both endpoints retained, every retained vertex sees zero on all incoming cells, so replay succeeds. Thus retained vertices are exactly independent sets, and removed vertices are exactly vertex covers. Minimizing collateral vertex removals is minimum vertex cover. This polynomial construction proves NP-hardness of that optimization task and gives a separate small exact oracle; it is not reported as a baseline speed comparison or as a new graph-complexity result.

Taken together, the results separate three distinct reasons to stop using a plain dependency graph. Alternative providers can violate intersection closure; inhibiting writers can violate union closure;

and combinations of locally short conditions can encode hard global choices. The next section's certificate interface preserves those distinctions: it accepts a direct successful replay when one exists, and demands explicit evidence when the producer claims a minimum or an impossibility.

## 6 Certificates and a Separate Executable Checker

The producer's output should reveal which guarantee it claims. A successful execution, a minimum local explanation, an optimal policy completion, and an infeasible policy are different propositions. Their certificates share the same admitted trace but have different quantifiers. The checker treats the producer, its compiled clauses, and its search history as untrusted.

Separating proof construction from checking is an established design principle, including in proof-carrying code [Necula 1997]. The present certificate language is much narrower: it validates finite captured-event semantics rather than machine-code safety. Its raw semantic cuts are not DRAT derivations, and no external SAT proof checker is executed. Clause-proof checking and trimming also do not by themselves establish minimum explanation size [Wetzler et al. 2014]. Here minimum claims require explicit additional checks.

### 6.1 The admitted input and the trusted surface

The executable input gives domain sizes, designated node cells, edge triples, the base state, a tag count, and an ordered event list. Domain members are encoded as integers; the meaning of a source attribute's integer codes is documented in its example. Guard maps contain explicit nonempty value sets. Write maps contain one constant per written cell. Duplicate cells, duplicate guard values, invalid domain members, out-of-range tags, and unexpected fields are rejected rather than silently normalized.

The checker independently validates the graph signature and base, the static event rule, and the full original trace. It does not accept an input merely because the producer already parsed it. Its Python integer checks distinguish actual integers from Boolean values, which otherwise behave as integers in some operations. Retention masks must be in range, and policy keep/remove masks must be disjoint. These are mundane checks with semantic consequences: accepting a high bit or duplicate write can change the proposition a packet appears to prove.

The bounded implementation admits at most 512 cells, 64 node cells, 192 edge cells, 256 values per domain, 12 tags, and 96 events. Refutation trees have depth at most the tag count and at most 10,000 counted nodes. The command-line reader also has a 16 MiB packet or decompressed-line limit. The structural bounds do not imply that every possible serialization fits that byte limit; oversized packets are outside the executable admission contract. No general theorem about unbounded inputs is inferred from these bounds.

The trusted computing base includes this checker, the interpreter, and the input bytes it is asked to check. The mathematical proof describes the intended judgments. It is not a machine-verified proof that the Python implementation refines those judgments. The checker uses only standard-library imports and does not call the engine, normalizer, producer, or independent oracle. That is code separation, not independent authorship or a claim of absence of correlated design mistakes.

### 6.2 Successful outcomes verify both directions

A successful outcome consists of a retained mask  $K$ , final state  $S_f$ , and one receipt for each retained occurrence in original order. The checker starts at  $B$ , tests each retained observation, and computes the update. At every step it requires the receipt's occurrence index, old-value domain, and old values to match the independently computed pre-state exactly. Singleton-inferable old values must not appear as redundant supplied coordinates.

After forward replay, the checker compares the computed final state with  $S_f$  and checks final typing. It then applies the inverse procedure in reverse occurrence order, checks inverse postconditions and restored observations, and requires exact recovery of  $B$ . Theorems 2.2–2.4 describe the resulting semantic guarantee. The second direction also tests that the receipt format and its decoder agree, rather than merely accepting a final-state calculation.

The old-value equality check is what rejects the stale receipt in Figure 1. A checker that only required a plausible old value from the cell’s domain would accept a different predecessor. A checker that only compared the final state would not see the error at all. Every accepted receipt is tied to the selected execution’s actual pre-state, which is the operational content of rebasing.

### 6.3 Raw semantic cuts

A cut is a small reason that a set of partial decisions cannot have a valid completion. It identifies an observation  $(i, x, S)$  and a source position  $j < i$ , or  $j = -1$  for the base. Its surrounding partial assignment  $(P, D)$  must force the reader kept. If  $j \neq -1$ , it must force the source’s tag kept, and that event must write an unacceptable value to  $x$ . A base source must likewise be unacceptable. Finally every write between the source and reader whose constant belongs to  $S$  must have its tag forced removed.

Later bad writers need not be absent. They can replace one unacceptable value with another, but cannot make the observation hold. A good write carrying a tag already forced kept prevents the cut from being accepted, even if the producer intends some other occurrence of that tag to be removed. Grouping is part of the check, not a discretionary interpretation of the proof.

LEMMA 6.1 (RAW CUT SOUNDNESS). *Every completion of an accepted raw cut’s partial assignment fails its selected ghost observation, and hence cannot be a valid replay.*

PROOF. The selected bad source supplies an unacceptable value and is present by its required fact, unless it is the base. Every later potentially repairing write is absent. Thus any later retained write is also bad, and the last value before the forced reader remains outside  $S$ . Equation (6) is false. Theorem 3.1 excludes successful replay.  $\square$

A cut does not require its source to be the last retained writer under every completion. That stronger requirement would introduce unnecessary decisions about later bad writers. It needs only a retained bad source and the absence of all later good writers. This makes cuts useful leaves of a search proof, even though they are not always the cardinality-minimum local explanations of Section 4.

### 6.4 Refutation trees and authorized cost leaves

A refutation tree has three node forms. A cut leaf contains the source and observation just described. A split node chooses a tag unassigned by its current policy and contains both children: one forces that tag removed and the other forces it kept. A cost leaf is allowed only when the checker has supplied an external optional-removal budget, and the current forced optional removals already exceed it. A producer cannot authorize a cost leaf by putting a preferred budget inside that leaf.

THEOREM 6.2 (REFUTATION SOUNDNESS AND FINITE COMPLETENESS). *An accepted tree proves that its root policy has no valid completion within its authorized budget, if any. Conversely, whenever no such completion exists, a finite tree of this form exists.*

PROOF. For soundness, induct on the accepted tree. A cut leaf is sound by Lemma 6.1. At a cost leaf, later assignments cannot undo a removal already forced, so no completion can fall within the budget. A split partitions all completions into its removal and retention children. By induction neither part contains an admissible completion.

For completeness, split on unassigned tags until the mask is full. If its cost exceeds the budget, use a cost leaf. Otherwise, by hypothesis it is invalid. Choose a failed ghost observation and its last retained writer, or its bad base if no writer is retained. That source is bad and every later good writer is absent. The full mask therefore supports a raw cut. This closes every leaf of a finite complete decision tree.  $\square$

With  $m$  tags a full binary tree has at most  $2^{m+1} - 1$  nodes, which is 8,191 at  $m = 12$ . This establishes a finite bounded certificate route, not a polynomial proof-size guarantee. A partial policy can shorten the tree, and a raw cut can close a branch before every tag is assigned. The producer uses these opportunities, but checker soundness does not depend on how effectively it searches for them.

For an outcome of collateral cost  $c$ , an optimality certificate contains that checked outcome and a refutation for all completions of cost at most  $c - 1$ . The checker computes  $c$  from the mask and policy, sets the budget itself, and validates the tree. A cost-zero outcome is handled uniformly: the better budget is  $-1$ , so a cost leaf can immediately establish that no strictly better cost exists. No running-time counter or producer assertion is accepted as a lower bound on the optimum.

## 6.5 Cardinality-minimum conflicting subpolicies

Let the requested policy  $Q$  be a set of signed facts. A conflicting subpolicy  $C \subseteq Q$  is one having no valid completion. Proving that  $C$  is inclusion-minimal requires feasibility after deleting each fact of  $C$ . Proving it is minimum cardinality *among subpolicies of  $Q$*  is stronger: a different smaller conflict may use facts outside  $C$  but still in  $Q$ .

Suppose the proposed core has  $k$  facts. Its certificate contains a refutation of  $C$  and a valid retained-mask witness for every  $(k - 1)$ -fact subpolicy of  $Q$ . The checker constructs the required subpolicy set itself. It checks that each witness extends its designated subpolicy and directly replays successfully; it rejects missing, repeated, or unrelated witnesses. These witnesses need only prove existence of a valid mask, so they do not carry complete inverse receipts.

**THEOREM 6.3 (MINIMUM POLICY OBSTRUCTION).** *An accepted core certificate proves that  $C$  is an infeasible subpolicy of  $Q$  and that no smaller subpolicy of  $Q$  is infeasible. Every cardinality-minimum core has witnesses of this form in the finite semantic system.*

**PROOF.** The tree proves  $C$  infeasible. Any subpolicy with fewer than  $k$  facts extends inside  $Q$  to one of size  $k - 1$ . The certificate supplies a valid witness for that extension, which also satisfies the smaller subpolicy. Thus no smaller conflict exists. Conversely, when  $C$  is minimum, every  $(k - 1)$ -fact subpolicy is feasible and has a witness. Finite refutation completeness supplies the tree for  $C$ .  $\square$

The number of required witnesses is  $\binom{|Q|}{k-1}$ , not merely  $k$ . At twelve tags it is at most 924. The checker limits the sum of tree nodes and witness records to 10,000; the coarse full-tree bound plus this witness bound is 9,115. This count describes logical proof nodes and masks, not serialized bytes or the cost of replaying each witness. Byte admission and execution budgets are checked separately.

The certificate states the universe over which minimum is claimed. A core minimum within  $Q$  need not be a smallest impossible policy that could be written using entirely different facts. Nor does a minimum number of facts minimize source-code edits, removed graph elements, or any weighted notion of disruption. Those would be different optimization objectives.

## 6.6 Checking local minimum cardinality without normalization

For a local certificate the checker does not use the producer's clauses or the last-occurrence normalizer. It evaluates the chosen raw ghost observation on all  $2^m$  masks. Relative to the queried invalid mask  $M$ , let

$$A[d] = \text{true} \iff H_o(M \triangle d),$$

where masks identify subsets and  $\triangle$  is symmetric difference. Initially  $A[d]$  records whether exactly that difference produces a good local completion. A subset-OR transform computes

$$A^\downarrow[z] = \bigvee_{d \subseteq z} A[d]. \quad (17)$$

For each bit  $b$ , replace  $A[z]$  by  $A[z] \vee A[z \setminus \{b\}]$  whenever  $b \in z$ . After processing a set of bit positions, the entry includes precisely the original differences obtained by deleting any subset of those processed positions. Induction over the bits proves (17).

A fact set  $S$  consistent with  $M$  permits variation exactly outside  $S$ . It forces failure if and only if  $A^\downarrow[\mathcal{A} \setminus S]$  is false. Enumerating all  $S$  and taking the smallest cardinality therefore independently verifies both that the submitted facts force failure and that no smaller consistent fact set does. This checker is exponential in  $m$  and intentionally limited to small admitted inputs. It gives a distinct bounded validation algorithm rather than an illusion that the local certificate itself contains a proof-assistant theorem.

## 6.7 What consistency checking cannot authenticate

A producer may alter a final value while leaving its trace unchanged; the checker rejects that inconsistency. If an adversary instead changes the base and receipts together to form a different internally valid trace, the checker may accept it. That is the correct result for its proposition: it has validated the packet it received. It has not proved that the packet came from a particular repository, real model, or earlier run.

The mutation suite includes this positive control explicitly. Its acceptance prevents a misleading security claim that all changed inputs are rejected. Input provenance, publication identity, and source-model correspondence require trusted acquisition or separate evidence. The source-guided examples document their abstractions rather than relying on the semantic checker to authenticate a published diagram.

Finally, certificates do not enlarge the language. A refusal about fixed matches does not become a refusal about all rematchings because it has been checked. A minimum fact set does not become a minimum graph residual because it is small. The checker reduces trust in search and data processing; it cannot repair an unstated or incorrect semantic contract.

## 7 Finite Validation and Its Interpretation

The experiments target the executable semantics, the normalizer, and the certificate boundary. They are not measurements of a deployed weaving tool or benchmarks of a proof assistant. The evidence is organized by the proposition being challenged, rather than by treating every generated query as an independent application example.

### 7.1 Protocol and distinct evidence populations

The first discriminating pilot enumerated admitted one-cell traces and compared every retained mask with direct replay. A separate pilot exercised non-singleton accepting sets and repeated-tag behavior. The hardest global search obligations were checked on small SAT and vertex-cover constructions against exact truth oracles. The subsequent 50,000-input campaign froze the generators,

deterministic masks, elementary ablations, and certificate selection. A separate structural follow-up tested the stronger expressivity and two-write arguments after their mathematical development. Its inputs are not added to the main denominator.

All experimental data are newly generated finite models or explicitly documented source-guided projections. There are no learned parameters, training examples, held-out model selection, private repositories, real devices, external compute, or human-participant observations. Seeds make the generated input population reproducible; they do not turn it into a statistically representative sample of model repositories. The generated traces are conditioned on original validity, which is a stated input assumption of the calculus.

Failure criteria were any disagreement between compilation and direct replay, any accepted inconsistent certificate, any missed or spurious local prime in small exhaustive checking, any smaller policy conflict than certified, any admitted update with an ill-typed post-state, or a measured resource violation. Negative controls are not removed to improve a success proportion. An early tuple-versus-list assertion mismatch in a test harness was repaired without changing the semantic engine; the delivered tests and clean reproduction use the corrected comparison.

## 7.2 What is exhaustively enumerated?

The exact tiny family has one Boolean cell, three declared aspect tags, and trace length from zero through three. At each prefix, the next event chooses one of three tags, either no observation or the singleton containing the current value, and either no write or a write of zero or one. Every event is enabled on the original trace by construction. There are 18 choices per extension and two base values, so the exact input count is

$$2(1 + 18 + 18^2 + 18^3) = 12,350. \quad (18)$$

Every input is checked on all eight masks, giving 98,800 retention queries. Syntactically different traces remain distinct even when semantically equivalent. Unused tags are permitted.

This is exhaustive enumeration of that precisely specified syntax family, not all graphs with eight nodes, all arbitrary observations, or all aspect sets up to twelve tags. Tautological observations are not a separate branch of this tiny generator. Non-singleton observations are exercised in the other pilots and scalar generators. Stating these dimensions is necessary because “all tiny models” without a syntax and admission rule has no reproducible count.

A second exact check targets local primes rather than trace enumeration. For 32 predetermined four-tag inputs, 358 recorded observations are compared against all  $3^4 = 81$  signed clause candidates and all  $2^4 = 16$  masks. Each variable may be absent, positive, or negative in a clause. The oracle checks implication and proper-subclause minimality directly. It does not call the last-occurrence normalizer. This attacks completeness of the prime family, not only truth of a few emitted clauses.

## 7.3 The main 50,000-input campaign

The main seed is 27,182,818. In addition to the tiny family, the campaign has 36,400 three-tag scalar cases with two cells, three events, and domain size three; 1,200 scalar cases with four through eight tags, three cells, eight events, and domain size three; and 50 graph-presence cases with 64 nodes, 192 edges, 64 scalar attributes, twelve tags, and 96 events. Graph attributes have four values. Node and edge cells remain Boolean.

The scalar generator independently decides whether to observe and write each cell, and its accepting set always includes the original pre-value. The graph generator mixes attribute updates, edge activation, edge removal, node activation, and node deletion with incident-edge removal. Edge activation either activates missing endpoints in the same event or observes existing endpoints

Table 1. Main campaign populations. “Invalid” is a direct-replay result, not an excluded input. Every input has a valid original full trace.

| Population            | Inputs | Queries | Invalid | Tags |
|-----------------------|--------|---------|---------|------|
| Exact one-cell syntax | 12,350 | 98,800  | 5,976   | 3    |
| Small scalar          | 36,400 | 127,401 | 6,808   | 3    |
| Medium scalar         | 1,200  | 4,670   | 1,384   | 4–8  |
| Graph presence        | 50     | 200     | 67      | 12   |
| Total                 | 50,000 | 231,071 | 14,235  |      |

present. These construction choices maintain the static type rule rather than relying on post hoc rejection of ill-typed cases.

Outside the exhaustive family, up to four predetermined masks are tested: full retention, one designated tag removed, and two masks from fixed arithmetic functions of the input index. Duplicate masks are removed before counting. These are not all masks for the larger cases. The total is 231,071 distinct within-input queries, of which 216,836 replay successfully and 14,235 fail. Compilation and the direct raw interpreter agree on every query.

Each case emits one certificate. Every twentieth eligible non-graph case with a queried invalid mask emits a minimum local obstruction for that mask; otherwise the first queried valid mask supplies a successful outcome. This produces 49,526 success certificates and 474 local certificates, all accepted by the separate checker. The compiled formulas contain 17,678 local clauses and 37,906 literal occurrences in total. These are aggregate output counts, not maximum per-observation sizes or a claim that every observation is nontrivial.

Certificate coverage is narrower than the input count might suggest. Of the 49,526 success certificates, 21,213 use the empty retention mask. In total 26,065 execute no retained occurrence, because some nonempty masks contain only unused tags. The other 23,461 check 49,861 occurrence receipts. Exactly 548 success certificates use the full tag set. Thus “one certificate per input” must not be read as 50,000 nonempty inverse executions. Every input’s full trace is still executed during admission, and the structured tests separately exercise nonempty receipts, stale values, and reversal order.

The campaign is consequently strong evidence against many small implementation mismatches, but not a substitute for the round-trip proof. It also does not exhaust global minimum-policy certificates on every generated case. Those exponential obligations are tested in the small policy and reduction suites, where exact independent answers can be enumerated.

#### 7.4 Information ablations and fair comparisons

The ablation population is all 12,350 tiny cases plus the first 2,000 small scalar cases, with the same masks and direct oracle. It contains 105,801 queries. Four elementary implementations deliberately omit different information. They are not ports of GeKo, lens systems, reversible event-structure tools, or state-of-the-art selective-undo systems.

The original-provider baseline records a binary requirement from every reader to its original last writer. It can conservatively reject a valid replacement provider. A write-only baseline ignores observations and always computes a ghost final state. A snapshot-delta baseline additionally omits writes that were idempotent in the original execution. A selective-old-log baseline starts from the original final state and reverses erased occurrences using their original old values and postcondition

Table 2. Elementary information ablations on 105,801 fixed queries. A wrong-state count concerns an originally valid queried replay that the baseline returns with a different state. It is not an additional false-accept category.

| Information retained             | False accept | False reject | Wrong state |
|----------------------------------|--------------|--------------|-------------|
| Original-provider requirements   | 0            | 6,367        | —           |
| Writes, no observations          | 6,338        | 0            | 0           |
| Original snapshot deltas         | 6,338        | 0            | 1,956       |
| Selective original old-value log | 4,034        | 6,469        | 2,085       |

checks. It does not rebase the retained execution. Full reverse-order rollback of the original trace is correct for its own task; the last baseline is specifically the unsupported selective use of that log.

The original-provider baseline’s zero observed false accepts has a semantic explanation: keeping the original last writer for every kept reader preserves its original observed value. Its false rejects exhibit alternative providers, not random classification noise. The write-only baseline cannot reject any invalid mask because it has no observation predicate. It gives the right final state whenever replay actually succeeds, since both apply the same writes. Removing idempotent writes can additionally change a valid selected outcome. These are information-loss counterexamples, not claims of a performance win.

False accepts, false rejects, and wrong states answer different questions. They must not be added into a single error rate when their conditions overlap or when one baseline does not produce a state. No confidence interval is reported as if the deterministic generated population sampled an external deployment randomly. The counts describe this population exactly.

## 7.5 Typing, commuting swaps, and certificate corruption

The semantic suite enumerates 8,640 event/state combinations on two nodes and one edge. Of these, 2,130 are admitted and enabled; none produces an ill-typed post-state. This tests the sufficient typing rule against all states and event choices in that small test family. It does not establish completeness of the rule for safe graph updates.

Thirty-two admitted independent swaps produce no difference in final state or corresponding complements. Named counterexamples exercise replacement, inhibition, grouped repeated writes, accepting-set versus equality observations, stale complements, order omission, and original idempotent deltas. Across eleven named inputs, all 124 masks and all 675 partial policies are checked. The local/global example confirms local size three versus global size two, including valid smaller-policy witnesses.

The checker rejects nineteen classes of inconsistent mutation: incorrect final state or old value; missing or duplicate old coordinates; wrong occurrence or missing receipt; Boolean or out-of-range masks; unknown fields or duplicate writes; out-of-domain values or invalid original guards; a nonforcing local explanation; missing minimum-core witnesses; an unforced bad source; a wrong cost; a missing proof branch; an unauthorized cost leaf; and a dangling base edge. A changed base and consistently changed receipt are deliberately accepted. That positive control limits the claim to internal consistency, not provenance authentication or rejection of every changed byte.

## 7.6 Structural follow-up and exact reduction oracles

The structural suite contains 165 additional inputs. It realizes all 46 distinct one- or two-negative mixed clause types over four variables, then 64 predetermined conjunctions of one through six such clauses. All sixteen masks are compared with the intended Boolean formula. It also checks 32

five-tag one-writer traces on all 32 masks and all  $3^5 = 243$  partial policies each. The resulting totals are 2,784 direct retention queries and 7,776 one-writer policy queries.

Eighteen two-writer SAT constructions use one through three variables and three clauses, compared with exhaustive truth assignments. The tests separately assert Boolean domains, one event per tag, at most two writer occurrences per cell, and local prime width at most three. Five choice-tree constructions use two through six leaves and check minimum cores through size seven. The strict four-variable nonrepresentation argument is checked against the finite mixed clause family. No disagreement occurs.

The earlier semantic suite additionally checks sixteen basic SAT constructions and sixteen vertex-cover constructions against exact oracles. These are different families from the eighteen strengthened two-writer cases and are reported separately. Their role is to expose an incorrect reduction implementation or a minimum-certificate error. The general complexity statements still rely on the polynomial constructions and both directions of their mathematical proofs.

### 7.7 Six source-guided projections, with visible abstraction limits

Six compact encodings cover five source figures or passages. An independent class-model addition projects one example of aspect unweaving into node, edge, and attribute cells. An additive example groups two match occurrences under one aspect tag; removing the aspect that supplied one match while forcing the whole later aspect to remain is infeasible in this grouped policy. The source's match-level unweaving can retain the unaffected match; our stronger grouping is not reported as a defect of the source algorithm [Klein et al. 2009].

A rematching example explicitly contrasts outcomes. Under fixed recorded matches, removing a deletion leaves one formerly hidden match untouched. Re-running source matching would transform that newly exposed match too. The encoded fixed-match state is intentionally different. Two threshold projections choose previous values 50 and 150 around the source's threshold 100 after a write of 200; those two old values are our finite representatives, not two additional published benchmarks. One policy fails and the other succeeds because membership in the accepting set changes.

The sixth projection encodes only the disjunctive enabling pattern from an event-structure example: either of two earlier providers enables a later event. It fixes one order and does not implement the source's entire reverse-transition semantics [Ulidowski et al. 2018]. Across these encodings all 28 masks agree with direct replay; four selected policies have checked optimal outcomes and two have checked minimum-conflict refusals. No published tool is executed.

These projections offer concrete interpretations and deliberate boundary cases. They do not establish production relevance, full source-model equivalence, or coverage of thirty independent published examples. Exact input JSON, source locations, code-value explanations, and abstraction boundaries are included so computational replay can be separated from evaluation of the literature interpretation.

### 7.8 Resources and reproducibility scope

All scientific runs use one worker under a four-core, 4 GiB, no-swap environment. The measured main campaign takes 24.1039 process CPU seconds and 24.1217 summed chunk wall seconds. Its process high-water RSS is 109,868 KiB, and its largest measured per-case interval is 0.0737 seconds. The campaign records 4,571,581 instrumented semantic, Boolean, oracle, and checking operations. These are named program counters, not all machine instructions.

The CPU interval includes chunk input construction, deterministic prefix skipping, checking, and serialization. Summed per-chunk timing excludes final aggregation overhead. The semantic suite takes 0.4821 CPU seconds, the structural suite 0.1317, and the six source projections 0.0018 in

their recorded runs. These numbers establish bounded feasibility, not a comparative speed claim. Operating system high-water RSS is a process measure and can include interpreter memory already present before the timed region.

Two earliest exploratory smoke invocations did not record CPU, and the earliest pilots did not record the full later primitive-counter vocabulary. Consequently, a completely measured cumulative CPU or primitive-operation total from the first exploration cannot be reconstructed and is not asserted. Later main, test, and clean-run records are retained separately. Runtime literature downloads failed without receiving input bytes; all executed data are original and bundled.

The standalone artifact requires only the Python standard library. A fresh reproduction reruns the pilots, semantic tests, structural suite, source projections, and main campaign, then independently checks all certificate streams. It compares uncompressed semantic records and scientific counters exactly while excluding machine-dependent timing fields. Resume-by-chunk metadata is a convenience, not an integrity proof; a clean run is the relevant reproduction route. The accompanying current-state record reports the actual clean-run outcome rather than treating a documented command as an executed result.

## 8 Relationship to Existing Interfaces and Scope

The relevant comparison is not whether another system uses a trace, a complement, a dependency, or a certificate. Those mechanisms have extensive histories. The comparison is the proposition that each interface preserves, and whether a result about that proposition transfers to grouped, fixed-match retention decisions.

### 8.1 Unweaving and the granularity of retained behavior

Aspect model unweaving provides the historical operational connection: a model's construction trace supports inverse operations and impact-sensitive undo, while some situations require new matching and reweaving [Klein et al. 2009]. Our input is instead the closed sequence after matching. We preserve its explicit observations, not equivalence to every re-execution of a source weaver. The source-guided rematching and additive examples make the distinction observable. Consequently, we do not attribute inverse tracing, value-sensitive impact, or independent/additive/general case analysis to the present work as new.

Grouping is another visible choice. A source-level aspect can have several match instances, and an application may prefer to remove some while keeping others. Our default policy deliberately keeps or removes all occurrences with one tag. Refining a tag into occurrence-specific tags changes the retention domain and can make an impossible grouped policy feasible. Theorems remain statements about the chosen input tags; a tool must not silently change their granularity to make a refusal disappear.

### 8.2 Bidirectional transformations and complement information

Lens laws organize relationships between a source and a view and specify how updates propagate [Foster et al. 2007]. Symmetric lenses make complement information explicit [Hofmann et al. 2011]. Our receipt-backed inverse uses the same basic idea of retaining information destroyed by a forward map. The coordinate necessity argument is correspondingly restricted to a product storage model, and is not a new general theory of least complements.

Relational lenses also place constraints such as functional dependencies inside a typed update interface [Bohannon et al. 2006]. Our graph-presence invariant is much narrower. It ensures endpoint presence for a fixed catalog but does not solve relational view-update policy or infer all consequences of a schema. The exact selective-replay question additionally fixes an ordered

set of observed event instances and asks which grouped decisions can remain. Neither a generic round-trip law nor an old-value log alone specifies that retention policy.

The distinction suggests how to compose interfaces without conflating them. An external capture layer could produce closed events and attest that its singleton observations cover all consumed inputs. The present calculus would then reason about retention within that captured trace. Establishing the capture layer’s soundness is an additional proof obligation. The article does not assume that every lens or graph weaver automatically provides it.

### 8.3 Causality, explanation, and representation

Reversible event structures already accommodate enabling relations richer than one fixed prerequisite per event [Ulidowski et al. 2018]. Axiomatic accounts also separate the assumptions needed for different reversible-computation properties [Lanese et al. 2024]. Our union/intersection criterion concerns the narrow representation by binary implications over retained tags; its failure is not a refutation of richer causal or event-structure representations. The footprint diamond is likewise one conservative sufficient condition, not a complete general causal semantics.

A recent reversible-logging preprint studies closure-preserving compression and rate-distortion with an explicit adequacy assumption for logged observables [Xu 2026]. Its optimization objective is not minimum retained-policy facts. In particular, Boolean encoding and closure reasoning can represent signed or negative enabling conditions; the existence of an inhibiting writer here does not by itself show that a closure-based encoding is impossible. Our impossibility result is specifically about *tag-to-tag implication* families, with union and intersection counterexamples.

Prime-implicate representations and their size tradeoffs belong to knowledge compilation [Darwiche and Marquis 2002]. Compatible prime reasons and distinctions between subset-minimal and shortest explanations belong to rigorous Boolean explanation [Darwiche and Hirth 2023; Darwiche and Ji 2022]. The last-writer normalization instantiates those ideas for one observation, and the exact  $M_2$  characterization and write-count boundary describe the resulting trace language. The article claims neither the invention of the prime-explanation principle nor a fastest general explanation algorithm.

The global results explain why the distinction matters. Local polarity can change from one observation to another. Their conjunction can therefore acquire new resolvents and shorter global explanations, or encode hard policy choices. A local minimum certificate is a useful precise diagnostic, but it is not a replacement for a global satisfiability or optimum certificate.

### 8.4 Limits of the result and evidence

The language excludes unrestricted imperative transformations, implicit reads, external side effects, fresh-name generation, arbitrary multi-cell predicates, rematching, and arbitrary semantic equivalence of models. An observation set is part of the fixed contract, not a post hoc relaxation. Graph updates are typed only for fixed endpoint presence; multiplicities, reachability, method semantics, and application-specific schema invariants require additional rules.

The complexity threshold counts writer occurrences per cell while allowing large event footprints. The local optimality bound concerns flat CNFs without auxiliary aspect variables. Minimum local facts, minimum conflicting subpolicies, and minimum collateral removals have distinct objective functions and quantifier universes. The checker is separately implemented but not mechanically verified, and finite tests cannot rule out all correlated bugs or establish a general proof-assistant theorem.

The evidence is bounded and predominantly synthetic. Only six source-guided projections are included, and they cover five passages rather than a broad published workload collection. Empty or unused-tag masks account for a substantial part of successful campaign certificates, as reported

explicitly. No production performance or user benefit is inferred from the measured seconds. Scholarly novelty and suitability for a particular publication standard require comparison beyond an internally consistent implementation and a collection of proofs.

## 8.5 Research assistance and artifact availability

Generative AI assistance was used substantively in formulating and comparing research directions, constructing and criticizing mathematical arguments, implementing the producer and checker, designing inputs and negative controls, executing and analyzing the bounded tests, preparing the manuscript, and conducting internal artifact review. The resulting code paths are separate implementations, not independently authored human validations. This disclosure does not assert human verification, authorship approval, or permission for external submission; those responsibilities remain to be established before external use.

The accompanying standalone artifact contains the event semantics, generator, producer, checker, exact finite inputs, mathematical proof notes, claim/evidence ledger, source attributions, raw results, and reproduction commands. No public repository address is asserted. No external service is needed to replay the bundled experimental inputs. The separate figure handoff concerns only possible visual finishing and does not delegate scientific proofs or experiments.

## 8.6 Conclusion

For grouped captured events, selective unweaving is not determined by final values or inverse logs alone. Explicit observations and ordered writes admit an exact local normalization: each observation has a unate prime-clause family and a minimum forcing explanation in a stated representation. Those local guarantees do not collapse the global policy problem. The language realizes exactly a mixed one/two-negative clause class; one write per cell yields implication closure, while two writes already yield NP-complete feasibility and arbitrarily large global conflicts despite ternary local clauses. Complement rebasing and separate certificate checks make the retained execution's round trip an explicit obligation. These conclusions apply to the defined observation-stable semantics, with the mathematical and finite evidence supporting different parts of that claim.

## References

- Aaron Bohannon, Benjamin C. Pierce, and Jeffrey A. Vaughan. 2006. Relational Lenses: A Language for Updatable Views. In *Proceedings of the 25th ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems*. ACM, 338–347. doi:10.1145/1142351.1142399
- Adnan Darwiche and Auguste Hirth. 2023. On the (Complete) Reasons Behind Decisions. *Journal of Logic, Language and Information* 32 (2023), 63–88. doi:10.1007/s10849-022-09377-8
- Adnan Darwiche and Chunxi Ji. 2022. On the Computation of Necessary and Sufficient Explanations. *Proceedings of the AAAI Conference on Artificial Intelligence* 36, 5 (2022), 5582–5591. doi:10.1609/aaai.v36i5.20498
- Adnan Darwiche and Pierre Marquis. 2002. A Knowledge Compilation Map. *Journal of Artificial Intelligence Research* 17 (2002), 229–264. doi:10.1613/jair.989
- J. Nathan Foster, Michael B. Greenwald, Jonathan T. Moore, Benjamin C. Pierce, and Alan Schmitt. 2007. Combinators for Bidirectional Tree Transformations: A Linguistic Approach to the View-Update Problem. *ACM Transactions on Programming Languages and Systems* 29, 3 (2007), 17. doi:10.1145/1232420.1232424
- Martin Hofmann, Benjamin C. Pierce, and Daniel Wagner. 2011. Symmetric Lenses. In *Proceedings of the 38th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. ACM, 371–384. doi:10.1145/1926385.1926428
- Alexey Ignatiev and João Marques-Silva. 2021. SAT-Based Rigorous Explanations for Decision Lists. In *Theory and Applications of Satisfiability Testing (Lecture Notes in Computer Science, Vol. 12831)*. Springer, 251–269. doi:10.1007/978-3-030-80223-3\_18
- Richard M. Karp. 1972. Reducibility among Combinatorial Problems. In *Complexity of Computer Computations*. Springer, 85–103. doi:10.1007/978-1-4684-2001-2\_9
- Jacques Klein, Jörg Kienzle, Brice Morin, and Jean-Marc Jézéquel. 2009. Aspect Model Unweaving. In *Model Driven Engineering Languages and Systems (Lecture Notes in Computer Science, Vol. 5795)*. Springer, 514–530. doi:10.1007/978-3-642-04425-0\_39

- Ivan Lanese, Iain Phillips, and Irek Ulidowski. 2024. An Axiomatic Theory for Reversible Computation. *ACM Transactions on Computational Logic* 25, 2, Article 11 (2024), 40 pages. doi:10.1145/3648474
- George C. Necula. 1997. Proof-Carrying Code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. ACM, 106–119. doi:10.1145/263699.263712
- Irek Ulidowski, Iain Phillips, and Shoji Yuen. 2018. Reversing Event Structures. *New Generation Computing* 36, 3 (2018), 281–306. doi:10.1007/s00354-018-0040-8
- Nathan Wetzler, Marijn J. H. Heule, and Jr. Warren A. Hunt. 2014. DRAT-trim: Efficient Checking and Trimming Using Expressive Clausal Proofs. In *Theory and Applications of Satisfiability Testing (Lecture Notes in Computer Science, Vol. 8561)*. Springer, 422–429. doi:10.1007/978-3-319-09284-3\_31
- Jianfeng Xu. 2026. Closure-Preserving Rate–Distortion for Reversible Logging. arXiv:2606.16592 doi:10.48550/arXiv.2606.16592 Preprint, second revision.